

Sistemas de Arquivos Distribuídos

- 12.1 Introdução
- 12.2 Arquitetura do serviço de arquivos
- 12.3 Estudo de caso: Sun Network File System
- 12.4 Estudo de caso: Andrew File System
- 12.5 Aprimoramentos e outros desenvolvimentos
- 12.6 Resumo

Um sistema de arquivos distribuído permite aos programas armazenarem e acessarem arquivos remotos exatamente como se fossem locais, possibilitando que os usuários acessem arquivos a partir de qualquer computador em uma rede. O desempenho e a segurança no acesso aos arquivos armazenados em um servidor devem ser comparáveis aos arquivos armazenados em discos locais.

Neste capítulo, definiremos uma arquitetura simples para sistemas de arquivos e descreveremos duas implementações básicas de serviços de arquivos distribuídos com projetos contrastantes que estão em pleno uso há duas décadas:

- o Network File System, NFS, da Sun;
- o Andrew File System, AFS.

Cada um deles simula a interface de sistema de arquivos do UNIX com diferentes graus de escalabilidade, tolerância a falhas e variações da restrita semântica de cópia única (*one-copy*) do UNIX.

Também serão examinados outros sistemas de arquivos que exploram novos modos de organização de dados no disco, ou entre vários servidores, para obter sistemas de arquivos de alto desempenho, tolerantes a falhas e com escalabilidade. Diferentes tipos de sistemas de armazenamento serão descritos em outras partes do livro. Isso inclui os sistemas de armazenamento *peer-to-peer* (Capítulo 10), os sistemas de arquivos replicados (Capítulo 18), os servidores de dados multimídia (Capítulo 20) e o estilo de serviço de armazenamento específico exigido para suportar pesquisas de busca na Internet e outras aplicações de grande escala com uso intensivo de dados (Capítulo 21).

12.1 Introdução

Nos Capítulos 1 e 2, identificamos o compartilhamento de recursos como um objetivo chave dos sistemas distribuídos. O compartilhamento de informações armazenadas talvez seja o aspecto mais importante dos recursos distribuídos. Os mecanismos para compartilhamento de dados assumem muitas formas e serão descritos em várias partes deste livro. Os servidores Web fornecem uma forma restrita de compartilhamento de dados, na qual os arquivos armazenados de forma local no servidor estão disponíveis para clientes em toda a Internet, mas os dados acessados por meio de servidores Web são gerenciados e atualizados em sistemas de arquivos no servidor, ou distribuídos em uma rede local. O projeto de sistemas de armazenamento de arquivos remotos para leitura e escrita em larga escala apresenta problemas de balanceamento de carga, confiabilidade, disponibilidade e segurança, cuja solução é o objetivo dos sistemas de armazenamento de arquivos *peer-to-peer*, descritos no Capítulo 10. O Capítulo 18 aborda os sistemas de armazenamento replicados, que são convenientes para aplicações que exigem acesso confiável a dados armazenados em sistemas nos quais a disponibilidade de computadores individuais não pode ser garantida. No Capítulo 20, descreveremos um servidor de mídia projetado para servir fluxos de dados de vídeo em tempo real para grandes números de usuários. O Capítulo 21 também descreve um sistema de arquivos para suportar aplicações de grande escala com uso intensivo de dados, como a pesquisa de busca na Internet.

Os requisitos de compartilhamento dentro de redes locais e intranets levam à necessidade de um tipo de serviço diferente – que suporte o armazenamento persistente dos dados e programas de todos os tipos e a distribuição consistente de dados atualizados. O objetivo deste capítulo é descrever a arquitetura e a implementação desses sistemas de arquivos distribuídos *básicos*. Usamos a palavra “básico” para denotar os sistemas de arquivos distribuídos cujo principal objetivo é simular a funcionalidade de um sistema de arquivos não distribuído para programas clientes executados em computadores remotos. Eles não mantêm várias réplicas persistentes dos arquivos, nem suportam as garantias de largura de banda e temporização exigidas para fluxos de dados multimídia – esses requisitos serão tratados em capítulos posteriores. Os sistemas de arquivos distribuídos básicos fornecem um apoio fundamental para computação organizacional baseada em intranets.

Os sistemas de arquivos foram originalmente desenvolvidos para sistemas de computadores centralizados e computadores *desktop* como um recurso do sistema operacional que fornece uma interface de programação conveniente para armazenamento em disco. Subsequentemente, eles adquiriram características como controle de acesso e mecanismos de proteção de arquivos, que os tornaram úteis para o compartilhamento de dados e programas. Os sistemas de arquivos distribuídos suportam o compartilhamento de informações por meio de arquivos e recursos de *hardware* com armazenamento persistente em toda uma intranet. Um serviço de arquivo bem projetado dá acesso a arquivos armazenados em um servidor com desempenho e confiabilidade semelhantes a (e, em alguns casos, melhores que) arquivos armazenados em discos locais. Seu projeto é adaptado para as características de desempenho e confiabilidade das redes locais e, portanto, eles são eficazes no fornecimento de armazenamento persistente compartilhado para uso em intranets. Os primeiros servidores de arquivos foram desenvolvidos por pesquisadores nos anos 70 [Birrell e Neeham 1980, Mitchell e Dion 1982, Leach *et al.* 1983], e o Sun Network File System foi disponibilizado no início dos anos 80 [Sandberg *et al.* 1985, Callaghan 1999].

Um serviço de arquivos permite que os programas armazenem e acessem arquivos remotos exatamente como se fossem locais, possibilitando que os usuários acessem seus arquivos a partir de qualquer computador em uma intranet. A concentração do armazena-

	<i>Compartilhamento</i>	<i>Persistência</i>	<i>Cache/réplicas distribuídas</i>	<i>Manutenção da consistência</i>	<i>Exemplo</i>
Memória principal	✗	✗	✗	1	RAM
Sistema de arquivos	✗	✓	✗	1	Sistema de arquivos UNIX
Sistema de arquivos distribuído	✓	✓	✓	✓	Sun NFS
Web	✓	✓	✓	✗	Servidor web
Memória compartilhada distribuída	✓	✗	✓	✓	Ivy (DSM, Cap. 6)
Objetos remotos (RMI/ORB)	✓	✗	✗	1	CORBA
Armazenamento de objetos persistentes	✓	✓	✗	1	Persistent State Service do CORBA
Sistema de armazenamento <i>peer-to-peer</i>	✓	✓	✓	2	OceanStore (Cap. 10)

Tipos de consistência:

1 : Uma cópia restrita. ✓ : Garantias ligeiramente mais fracas. **2** : Garantias consideravelmente mais fracas.

Figura 12.1 Sistemas de armazenamento e suas propriedades.

mento persistente em alguns poucos servidores reduz a necessidade de armazenamento em disco local e (o mais importante) permite que sejam feitas economias no gerenciamento e no arquivamento (*backups*) dos dados persistentes pertencentes a uma organização. Outros serviços, como o de nomes, o de autenticação de usuário e o de impressão, podem ser mais facilmente implementados quando são capazes de usar o serviço de arquivos para atender a suas necessidades de armazenamento persistente. Os servidores Web confiam no sistema de arquivos para o armazenamento das páginas que servem. Nas organizações que operam servidores Web para acesso externo e interno por meio de uma intranet, esses servidores frequentemente armazenam e acessam o material a partir de um sistema de arquivos distribuído local.

Com o advento da programação orientada a objetos distribuída, surgiu a necessidade do armazenamento persistente e da distribuição de objetos compartilhados. Uma maneira de obter isso é dispor de objetos serializados (da maneira descrita na Seção 4.3.2) e armazenar e recuperar esses objetos usando arquivos. Contudo, esse método de obtenção de persistência e distribuição se torna impraticável para objetos que mudam rapidamente e, portanto, várias estratégias mais diretas foram desenvolvidas. A invocação a objeto remoto da linguagem Java e os ORBs do CORBA fornecem acesso a objetos remotos compartilhados, mas nenhum deles garante a persistência dos objetos, nem a replicação dos objetos distribuídos.

A Figura 12.1 apresenta um panorama dos tipos de sistema de armazenamento. Além daqueles já mencionados, a tabela inclui sistemas de memória compartilhada distribuída (DSM, Distributed Shared Memory) e armazenamento de objetos persistentes. A DSM foi descrita em detalhes no Capítulo 6. Ela fornece uma simulação de uma memória compartilhada por meio da replicação de páginas ou de segmentos de memória em cada computador (nó). Ela não fornece necessariamente persistência automática. O armazenamento de objetos persistentes foi apresentado no Capítulo 5 e seu objetivo é fornecer persistência para objetos compartilhados distribuídos. Exemplos incluem o Persistent State Service CORBA (veja o Capítulo 8) e extensões persistentes para Java [Jordan 1996, java.sun.com VIII]. Alguns projetos de pesquisa foram desenvolvidos em plataformas que suportam a replicação automática e o armazenamento persistente de objetos (por exemplo, PerDiS [Ferreira

Módulo de diretório:	relaciona nomes de arquivo com IDs de arquivo
Módulo de arquivo:	relaciona IDs de arquivo com arquivos em particular
Módulo de controle de acesso:	verifica a permissão para a operação solicitada
Módulo de acesso a arquivo:	lê ou escreve dados ou atributos
Módulo de bloco:	acessa e aloca blocos de disco
Módulo de dispositivo:	E/S de disco e uso de <i>buffers</i>

Figura 12.2 Módulos do sistema de arquivos.

et al. 2000] e Khazana [Carter *et al.* 1998]). Os sistemas de armazenamento *peer-to-peer* oferecem escalabilidade para suportar cargas de cliente muito maiores do que os sistemas descritos neste capítulo, mas acarretam altos custos de desempenho para fornecer controle de acesso seguro e consistência entre as réplicas que podem ser atualizadas.

A coluna *consistência* indica se existem mecanismos para a manutenção da consistência entre várias cópias dos dados quando ocorrem atualizações. Praticamente todos os sistemas de armazenamento contam com o uso de cache para otimizar o desempenho dos programas. O uso de cache foi aplicado pela primeira vez na memória principal e em sistemas de arquivos não distribuídos, e para eles, a consistência é estrita (denotada por 1, significando consistência de cópia única, na Figura 12.1) – após uma atualização, os programas não conseguem observar quaisquer discrepâncias entre as cópias na cache e os dados armazenados. Quando são usadas réplicas distribuídas, é mais difícil obter a consistência estrita. Os sistemas de arquivos distribuídos, como o Sun NFS e o Andrew File System, colocam em cache, nos computadores clientes, cópias parciais de arquivos e adotam mecanismos de consistência específicos para manter uma aproximação da consistência estrita. Isso está indicado por um tique (✓) na coluna da consistência na Figura 12.1 – discutiremos esses mecanismos, e até que ponto eles se desviam da consistência estrita, nas Seções 12.3 e 12.4.

A Web usa cache extensivamente, tanto em computadores clientes como em servidores *proxies* mantidos por organizações de usuários. A consistência entre o original e as cópias armazenadas em servidores *proxies* Web e em caches de clientes é mantida apenas por ações explícitas do usuário. Os clientes não são notificados quando uma página original, armazenada em um servidor, é atualizada; eles precisam fazer verificações explícitas para manter suas cópias locais atualizadas. Isso tem como objetivo a navegação adequada na Web, mas não suporta o desenvolvimento de aplicativos cooperativos, como o do quadro branco compartilhado distribuído. Os mecanismos de consistência usados nos sistemas DSM são discutidos em detalhes no *site* que acompanha o livro [www.cdk5.net] (em inglês). Os sistemas de objetos persistentes variam consideravelmente em sua estratégia para uso de cache e consistência. Os esquemas CORBA e Persistent Java mantêm cópias únicas de objetos persistentes e é necessário usar invocação remota para acessá-los; portanto, o único problema de consistência se dá entre a cópia persistente de um objeto no disco e a cópia ativa na memória, que não é visível para clientes remotos. Os projetos PerDiS e Khazana, mencionados anteriormente, mantêm réplicas dos objetos em cache e empregam mecanismos de consistência bastante elaborados para reproduzir àquelas encontradas nos sistemas DSM.

Apresentamos aqui alguns problemas mais amplos relacionados ao armazenamento e à distribuição de dados persistentes e não persistentes, agora vamos voltar ao assunto principal deste capítulo – o projeto de sistemas de arquivos distribuídos básicos. Descreveremos algumas características relevantes dos sistemas de arquivos (não distribuídos), na Seção 12.1.1, e os requisitos dos sistemas de arquivos distribuídos, na Seção 12.1.2.

Tamanho do arquivo
Horário de criação
Horário de acesso (leitura)
Horário de modificação (escrita)
Horário de alteração de atributo
Contagem de referência
Proprietário
Tipo de arquivo
Lista de controle de acesso

Figura 12.3 Estrutura básica de um registro de atributo de arquivo.

A Seção 12.1.3 apresentará os estudos de caso que serão utilizados por todo o capítulo. Na Seção 12.2, definiremos um modelo abstrato de um sistema de arquivos distribuído básico, incluindo um conjunto de interfaces de programação. O sistema Sun NFS será descrito na Seção 12.3; ele compartilha muitas das características do modelo abstrato. Na Seção 12.4, descreveremos o Andrew File System – um sistema amplamente usado, que emprega mecanismos de cache e de consistência substancialmente diferentes. A Seção 12.5 examina alguns desenvolvimentos recentes no projeto de serviços de arquivo.

Os sistemas descritos neste capítulo não cobrem todo o espectro dos sistemas de gerenciamento de arquivos e dados distribuídos. Vários sistemas, com características mais avançadas, serão descritos posteriormente neste livro. O Capítulo 18 contém uma descrição do Coda, um sistema de arquivos distribuído que mantém réplicas persistentes dos arquivos para obter confiabilidade, disponibilidade e trabalho em modo não conectado. O Bayou, um sistema de gerenciamento de dados distribuído que fornece uma forma de replicação com consistência fraca para obter alta disponibilidade, também será abordado no Capítulo 18. O Capítulo 20 abordará o servidor de arquivos de vídeo Tiger, projetado para fornecer a distribuição de fluxos de dados para grandes números de clientes. O Capítulo 21 descreve o GFS (Google File System), um sistema de arquivos projetado especificamente para suportar aplicações de grande escala com uso intensivo de dados, incluindo a busca na Internet.

12.1.1 Características dos sistemas de arquivos

Os sistemas de arquivos são responsáveis pela organização, armazenamento, recuperação, atribuição de nomes, compartilhamento e proteção de arquivos. Eles fornecem uma interface de programação que caracteriza a abstração de arquivo, liberando os programadores da preocupação com os detalhes da alocação e do leiaute do armazenamento físico no disco. Os arquivos são armazenados em discos ou em outra mídia de armazenamento não volátil.

Os arquivos contêm *dados* e *atributos*. Os dados consistem em uma sequência de elementos (normalmente, bytes – 8 bits), acessíveis pelas operações de leitura e escrita de qualquer parte dessa sequência. Os atributos são mantidos como um único registro, contendo informações como o tamanho do arquivo, carimbos de tempo, tipo de arquivo, identidade do proprietário e listas de controle de acesso. Uma estrutura de registro de atributo típica está ilustrada na Figura 12.3. Os atributos escondidos (*shadow attributes*) são gerenciados pelo sistema de arquivos e, normalmente, não podem ser atualizados por programas de usuário.

<i>filedes</i> = <i>open</i> (<i>nome</i> , <i>modo</i>)	Abre um arquivo existente com o <i>nome</i> fornecido.
<i>filedes</i> = <i>creat</i> (<i>nome</i> , <i>modo</i>)	Cria um novo arquivo com o <i>nome</i> fornecido.
	As duas operações produzem um descritor de arquivo referenciando o arquivo aberto. O <i>modo</i> é <i>read</i> , <i>write</i> ou ambos.
<i>status</i> = <i>close</i> (<i>filedes</i>)	Fecha o arquivo aberto referenciado por <i>filedes</i> .
<i>count</i> = <i>read</i> (<i>filedes</i> , <i>buffer</i> , <i>n</i>)	Transfere para <i>buffer</i> <i>n</i> bytes do arquivo referenciado por <i>filedes</i> .
<i>count</i> = <i>write</i> (<i>filedes</i> , <i>buffer</i> , <i>n</i>)	Transfere <i>n</i> bytes de <i>buffer</i> para o arquivo referenciado por <i>filedes</i> .
	As duas operações produzem o número de bytes realmente transferidos e avançam o ponteiro de leitura e escrita.
<i>pos</i> = <i>lseek</i> (<i>filedes</i> , <i>offset</i> , <i>whence</i>)	Desloca o ponteiro de leitura e escrita para <i>offset</i> (relativo ou absoluto, dependendo de <i>whence</i>).
<i>status</i> = <i>unlink</i> (<i>nome</i>)	Remove o arquivo <i>nome</i> da estrutura de diretório. Se o arquivo não tiver outras referências, ele será excluído.
<i>status</i> = <i>link</i> (<i>nome1</i> , <i>nome2</i>)	Cria uma nova referência, ou nome (<i>nome2</i>), para um arquivo (<i>nome1</i>).
<i>status</i> = <i>stat</i> (<i>nome</i> , <i>buffer</i>)	Escreve os atributos do arquivo <i>nome</i> em <i>buffer</i> .

Figura 12.4 Operações do sistema de arquivos UNIX.

Os sistemas de arquivos são projetados para armazenar e gerenciar um grande número de arquivos, com recursos para criação, atribuição de nomes e exclusão de arquivos. A atribuição de nomes de arquivos é suportada pelo uso de diretórios. Um *diretório* é um arquivo, frequentemente de um tipo especial, que fornece um mapeamento dos nomes textuais para identificadores internos de arquivo. Os diretórios podem incluir nomes de outros diretórios, levando ao conhecido esquema de atribuição hierárquica de nomes e aos *nomes de caminho* (*pathname*), usados no sistema de arquivo UNIX e em outros sistemas operacionais. Os sistemas de arquivos também assumem a responsabilidade pelo controle do acesso aos arquivos, restringindo o acesso de acordo com as autorizações dos usuários e com o tipo de acesso solicitado (leitura, atualização, execução, etc.).

O termo *metadados* é usado frequentemente para se referir a todas as informações extras armazenadas por um sistema de arquivos que são necessárias para o gerenciamento dos arquivos. Isso inclui atributos de arquivo, diretórios e todas as outras informações persistentes utilizadas pelo sistema de arquivos.

A Figura 12.2 mostra uma estrutura modular em camadas, típica, para a implementação de um sistema de arquivos não distribuído em um sistema operacional convencional. Cada camada depende apenas das camadas que estão abaixo dela. A implementação de um serviço de arquivo distribuído precisa de todos os componentes mostrados, com componentes adicionais para tratar da comunicação cliente-servidor e da atribuição de nomes e da localização de arquivos distribuídos.

Operações do sistema de arquivos • A Figura 12.4 resume as principais operações sobre arquivos que estão disponíveis nos sistemas UNIX para aplicativos. Essas são as chamadas de sistema implementadas pelo núcleo; normalmente, os programadores de aplicativos as acessam por meio de funções de biblioteca, como a biblioteca de entrada e saída padrão da linguagem C, ou as classes de arquivo Java. Fornecemos as primitivas como uma indicação das operações que os serviços de arquivo devem suportar e para comparação com as interfaces do serviço de arquivos que vamos apresentar a seguir.

As operações UNIX são baseadas em um modelo de programação no qual algumas informações sobre o estado do arquivo são armazenadas pelo sistema de arquivos para cada programa em execução.

Elas consistem, entre outros, em uma lista de arquivos correntemente abertos e um ponteiro de leitura e escrita que fornece a posição dentro do arquivo na qual será aplicada a próxima operação de leitura ou de escrita.

O sistema de arquivos é responsável por realizar o controle de acesso para os arquivos. Em sistemas de arquivos locais, como o UNIX, ele faz isso quando cada arquivo é aberto, verificando os direitos permitidos à identidade do usuário na lista de controle de acesso em relação ao *modo* de acesso solicitado na chamada de sistema *open*. Se os direitos corresponderem ao modo, o arquivo será aberto, e o *modo* é armazenado nas informações de estado de arquivo aberto.

12.1.2 Requisitos do sistema de arquivos distribuído

Muitos dos requisitos e das armadilhas em potencial no projeto de serviços distribuídos foram detectados pela primeira vez no desenvolvimento inicial dos sistemas de arquivos distribuídos. Inicialmente, eles ofereciam transparência de acesso e de localização; surgiram requisitos de desempenho, escalabilidade, controle de concorrência, tolerância a falhas e segurança, e eles foram atendidos em fases subsequentes de desenvolvimento. Vamos discutir esses e outros requisitos nas subseções a seguir.

Transparência • O serviço de arquivo normalmente é o mais usado em uma intranet; portanto, sua funcionalidade e seu desempenho são críticos. O projeto do serviço de arquivos deve suportar muitos dos requisitos de transparência dos sistemas distribuídos, identificados na Seção 1.5.7. O projeto deve contrabalançar a flexibilidade e a escalabilidade derivadas da transparência, com a complexidade e o desempenho do *software*. As seguintes formas de transparência são parcialmente, ou totalmente, tratadas pelos serviços de arquivos atuais:

Transparência do acesso: os programas clientes não devem conhecer a distribuição de arquivos. Um único conjunto de operações é fornecido para acesso a arquivos locais e remotos. Os programas escritos para operar sobre arquivos locais são capazes de acessar arquivos remotos sem modificação.

Transparência de localização: os programas clientes devem ver um espaço de nomes de arquivos uniforme. Os arquivos, ou grupos de arquivos, podem ser deslocados de um servidor a outro sem alteração de seus nomes de caminho, e os programas de usuário devem ver o mesmo espaço de nomes onde quer que sejam executados.

Transparência de mobilidade: nem os programas clientes, nem as tabelas de administração de sistema nos computadores clientes precisam ser alterados quando os arquivos são movidos. Isso permite a mobilidade do arquivo – arquivos ou, mais comumente, conjuntos ou volumes de arquivos podem ser movidos, ou pelos administradores de sistema ou automaticamente.

Transparência de desempenho: os programas clientes devem continuar a funcionar satisfatoriamente, enquanto a carga sobre o serviço varia dentro de um intervalo especificado.

Transparência de mudança de escala: o serviço pode ser expandido de forma paulatina, para lidar com uma ampla variedade de cargas e tamanhos de rede.

Atualizações concorrentes de arquivos • As alterações feitas em um arquivo por um único cliente não devem interferir na operação de outros clientes que estejam acessando, ou

alterando, o mesmo arquivo simultaneamente. Esse é o conhecido problema do controle de concorrência, discutido em detalhes no Capítulo 16. Em muitos aplicativos, a necessidade de controle de concorrência para acesso a dados compartilhados é amplamente aceita, e são conhecidas técnicas para sua implementação, mas elas são dispendiosas. A maior parte dos serviços de arquivo atuais segue os padrões UNIX modernos, fornecendo travamento (*locking*) em nível de arquivo ou em nível de registro.

Replicação de arquivos • Em um serviço de arquivos que suporta replicação, um arquivo pode ser representado por várias cópias de seu conteúdo em diferentes locais. Isso tem duas vantagens – permite que vários servidores compartilhem a carga do fornecimento de um serviço para clientes que acessam o mesmo conjunto de arquivos, melhorando a escalabilidade do serviço, e melhora a tolerância a falhas, permitindo que, em caso de falhas, os clientes localizem outro servidor que contenha uma cópia do arquivo. Poucos serviços de arquivo suportam replicação completa, mas a maioria suporta o armazenamento de arquivos, ou de porções de arquivos, em caches locais, que é uma forma limitada de replicação. A replicação de dados será discutida no Capítulo 18, que inclui uma descrição do serviço de arquivos replicado Coda.

Heterogeneidade do hardware e do sistema operacional • As interfaces de serviço devem ser definidas de modo que o *software* cliente e servidor possa ser implementado para diferentes sistemas operacionais e computadores. Esse requisito é um aspecto importante de sistemas abertos.

Tolerância a falhas • Por ser parte essencial nos sistemas distribuídos, é essencial que o serviço de arquivo distribuído continue a funcionar diante de falhas de clientes e servidores. Felizmente, um projeto moderadamente tolerante a falhas é fácil para servidores simples. Para suportar falhas de comunicação transientes, o projeto pode ser baseado na semântica de invocação *no máximo uma vez* (veja a Seção 5.3.1). Ou, então, ele pode usar uma semântica mais simples, como *pelo menos uma vez*, com um protocolo de servidor projetado em termos de operações *idempotentes*, garantindo que pedidos duplicados não resultem em atualizações inválidas nos arquivos. Os servidores podem ser *sem estado* (*stateless*), para que após uma falha o serviço possa ser reiniciado e restaurado sem necessidade de recuperar o estado anterior. A tolerância a falha de desconexão, ou de servidor, exige replicação de arquivo, a qual é mais difícil de obter (e será discutida no Capítulo 18).

Consistência • Os sistemas de arquivos convencionais, como o fornecido no UNIX, oferecem *semântica de atualização de cópia única* (*one-copy*). Isso se refere a um modelo de acesso concorrente a arquivos, no qual o conteúdo do arquivo visto por todos os processos que estão acessando, ou atualizando determinado arquivo, é aquele que eles veriam se existisse apenas uma cópia do conteúdo do arquivo. Quando os arquivos são replicados, ou armazenados em cache, em diferentes *sites*, há um atraso inevitável na propagação das modificações feitas em um *site* para os outros *sites* que contêm cópias, e isso pode resultar em certo desvio da semântica de cópia única.

Segurança • Praticamente todos os sistemas de arquivos fornecem mecanismos de controle de acesso baseados no uso de listas de controle de acesso. Nos sistemas de arquivos distribuídos, há necessidade de autenticar as requisições dos clientes para que o controle de acesso no servidor seja baseado nas identidades corretas de usuário e para proteger o conteúdo das mensagens de requisição-resposta com assinaturas digitais e (opcionalmente) criptografia de dados secretos. Vamos discutir o impacto desses requisitos em nossas descrições de estudo de caso.

Eficiência • Um serviço de arquivo distribuído deve oferecer recursos que tenham pelo menos o mesmo poder e generalidade daqueles encontrados nos sistemas de arquivos

convencionais, e deve obter um nível de desempenho comparável. Birrell e Needham [1980] expressaram seus objetivos de projeto para o CFS (Cambridge File Server) nos seguintes termos:

Desejaríamos ter um servidor de arquivos simples, de baixo nível, para compartilhar um recurso dispendioso, a saber, o disco, que nos deixasse livres para projetar um sistema de arquivos mais apropriado para um cliente em particular, ao mesmo tempo em que disponibilizasse um sistema de alto nível compartilhado entre os clientes.

A queda no custo do armazenamento em disco reduziu a importância do primeiro objetivo, mas a percepção da necessidade de uma gama de serviços tratando dos requisitos dos clientes com diferentes objetivos permanece e pode ser melhor resolvida por meio de uma arquitetura modular, do tipo esboçado anteriormente.

As técnicas usadas para a implementação de serviços de arquivos representam uma parte importante do projeto de sistemas distribuídos. Um sistema de arquivos distribuído deve fornecer um serviço que seja comparável aos sistemas de arquivos locais (ou melhor que eles), em termos de desempenho e confiabilidade. Ele deve ser conveniente para administrar, com operações e ferramentas que permitam aos administradores de sistema instalá-lo e operá-lo adequadamente.

12.1.3 Estudos de caso

Construiremos um modelo abstrato para um serviço de arquivos, para atuar como um exemplo introdutório, separando as preocupações com a implementação e fornecendo um modelo simplificado. Descreveremos o Sun Network File System com alguns detalhes, recorrendo ao nosso modelo abstrato mais simples para esclarecer sua arquitetura. O Andrew File System será descrito em seguida, fornecendo uma visão de um sistema de arquivos distribuído que adota uma estratégia diferente para escalabilidade e manutenção da consistência.

Arquitetura do serviço de arquivos • Trata-se de um modelo abstrato de arquitetura que serve tanto para o NFS como para o AFS. Ele é baseado em uma divisão de responsabilidades entre três módulos – um módulo cliente, que simula uma interface de sistema de arquivos convencional para programas aplicativos, e dois módulos servidores que efetuam operações em diretórios e em arquivos. A arquitetura é projetada para permitir uma implementação *sem estado* (*stateless*) dos módulos servidores.

Sun NFS • O NFS (Network File System) da Sun Microsystems foi amplamente adotado na indústria e nos ambientes acadêmicos, desde sua introdução, em 1985. O projeto e desenvolvimento do NFS foram feitos pelo pessoal da Sun Microsystems, em 1984 [Sandberg *et al.* 1985; Sandberg 1987, Callaghan 1999]. Embora vários serviços de arquivo distribuídos já tivessem sido desenvolvidos, e usados em universidades e laboratórios de pesquisa, o NFS foi o primeiro serviço de arquivo projetado como um produto. O projeto e a implementação do NFS obtiveram sucesso tanto técnico como comercial.

Para estimular sua adoção como padrão, as definições das principais interfaces foram colocadas em domínio público [Sun 1989], permitindo que outros fornecedores produzissem implementações, e o código-fonte de uma implementação de referência foi disponibilizado, sob licença, para outros fabricantes de computadores. Atualmente, ele é suportado por muitos fornecedores, e o protocolo NFS (versão 3) é um padrão da Internet, definido no RFC 1813 [Callaghan *et al.* 1995]. O livro de Callaghan sobre o NFS [Callaghan 1999] é uma excelente fonte sobre o projeto e desenvolvimento do NFS e a respeito de tópicos relacionados.

O NFS fornece acesso transparente a arquivos remotos para programas clientes executando em UNIX e outros sistemas operacionais. O relacionamento cliente-servidor é simétrico: cada computador em uma rede NFS pode atuar como cliente e como servi-

dor, e os arquivos de cada máquina podem se tornar disponíveis para acesso remoto por outras máquinas. Qualquer computador pode ser um servidor, exportando alguns de seus arquivos, e cliente, acessando arquivos de outras máquinas. No entanto, é uma prática comum, em instalações maiores, configurar algumas máquinas como servidores dedicados e outras como estações de trabalho.

Um objetivo importante do NFS é obter um alto nível de suporte para heterogeneidade de *hardware* e sistema operacional. O projeto é independente do sistema operacional: existem implementações de cliente e servidor para quase todos os sistemas operacionais e plataformas, incluindo todas as versões do Windows, Mac OS, Linux e qualquer outra versão de UNIX. Implementações do NFS em multiprocessadores de alto desempenho foram desenvolvidas por diversos fornecedores e são usadas para satisfazer os requisitos de armazenamento em intranets com muitos usuários concomitantes.

Andrew File System • O Andrew é um ambiente de computação distribuída, desenvolvido na Carnegie Mellon University (CMU) para uso como sistema de computação e informação do campus [Morris *et al.* 1986]. O projeto do Andrew File System (daqui em diante abreviado como AFS) reflete a intenção de suportar o compartilhamento de informações em larga escala, minimizando a comunicação cliente-servidor. Isso foi conseguido pela transferência de arquivos inteiros entre computadores servidores e clientes, armazenando-os em cache nos clientes até que o servidor receba uma versão mais atualizada. Vamos descrever o AFS-2, a primeira implementação de “produção”, seguindo as descrições de Satyanarayanan [1989a; 1989b]. Descrições mais recentes podem ser encontradas em Campbell [1997] e [Linux AFS].

Inicialmente, o AFS foi implementado na CMU em uma rede de estações de trabalho e servidores executando UNIX BSD e o sistema operacional Mach e, posteriormente, tornou-se disponível em versões comerciais e de domínio público. Uma implementação de domínio público do AFS está disponível no sistema operacional Linux [Linux AFS]. O AFS foi adotado como a base do sistema de arquivos DCE/DFS no DCE (Distributed Computing Environment) da Open Software Foundation [www.opengroup.org]. O projeto do DCE/DFS foi além do AFS sob vários aspectos importantes, os quais destacaremos na Seção 12.5.

12.2 Arquitetura do serviço de arquivos

Uma arquitetura que apresenta uma separação clara das principais preocupações no fornecimento de acesso aos arquivos é obtida por meio da estruturação do serviço de arquivo em três componentes – um *serviço de arquivos plano*, um *serviço de diretório* e um *módulo cliente*. Os módulos relevantes e seus relacionamentos aparecem na Figura 12.5. O serviço de arquivos plano e o serviço de diretório exportam uma interface para uso dos programas clientes, e suas interfaces RPC, consideradas em grupo, fornecem um amplo conjunto de operações para acesso aos arquivos. O módulo cliente fornece uma única interface de programação, com operações sobre arquivos semelhantes àquelas encontradas nos sistemas de arquivos convencionais. O projeto é *aberto*, no sentido de que vários módulos clientes podem ser usados para implementar distintas interfaces de programação, simulando as operações de arquivo de vários sistemas operacionais e otimizando o desempenho para diferentes configurações de *hardware* de cliente e servidor.

A divisão de responsabilidades entre os módulos pode ser definida como segue:

Serviço de arquivos plano (flat file service) • O serviço de arquivos plano se preocupa com a implementação de operações sobre o conteúdo dos arquivos. São usados *identificadores exclusivos de arquivo* (UFID, *Unique File IDentifiers*) para fazer referência aos

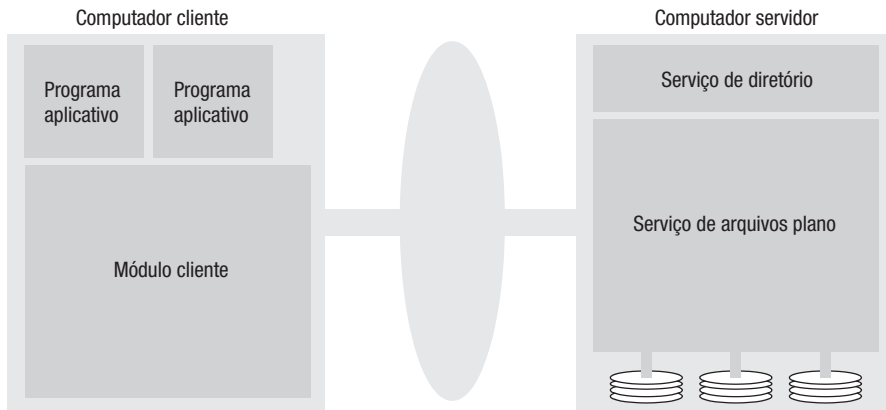


Figura 12.5 Arquitetura do serviço de arquivos.

arquivos em todas as requisições de operações ao serviço de arquivos plano. A divisão de responsabilidades entre o serviço de arquivos e o serviço de diretório é baseada no uso de UFIDs. As UFIDs são longas sequências de bits, escolhidas de modo que cada arquivo tenha um UFID único dentre todos os arquivos de um sistema distribuído. Quando o serviço de arquivos plano recebe uma requisição para criar um arquivo, ele gera um novo UFID e retorna esse UFID para o solicitante.

Serviço de diretório • O serviço de diretório fornece um mapeamento entre *nomes textuais* de arquivos e seus UFIDs. Os clientes podem obter o UFID de um arquivo fornecendo seu nome textual para o serviço de diretório. O serviço de diretório possui as funções necessárias para gerar diretórios, adicionar novos nomes de arquivo a eles e para obter suas UFIDs. Ele é um cliente do serviço de arquivos plano; os arquivos de seu diretório são armazenados em arquivos do serviço de arquivos plano. Quando um esquema de atribuição de nomes hierárquico é adotado, como no UNIX, os diretórios contêm referências para outros diretórios.

Módulo cliente • Um módulo cliente é executado em cada computador cliente, integrando e estendendo as operações do serviço de arquivos plano e do serviço de diretório sob uma interface de programação de aplicativo única, disponível para programas em nível de usuário nos computadores clientes. Por exemplo, em máquinas UNIX, seria fornecido um módulo cliente que simularia o conjunto completo de operações de arquivos UNIX, interpretando nomes de arquivos por meio de requisições iterativas ao serviço de diretório. O módulo cliente também contém informações sobre os locais de rede dos processos servidor de arquivos plano e servidor de diretório. Finalmente, o módulo cliente pode assumir um papel importante na obtenção de um desempenho satisfatório, por meio da implementação de uma cache de blocos de arquivo recentemente usados no cliente.

Interface do serviço de arquivos plano • A Figura 12.6 contém uma definição da interface para um serviço de arquivos planos. Essa é a interface RPC usada pelos módulos clientes. Normalmente, ela não é usada diretamente por programas em nível de usuário. Um argumento *FileId* é inválido se o arquivo a que se refere não está presente no servidor que processa a requisição, ou se suas permissões de acesso são inadequadas para a operação solicitada. Todas as funções da interface, exceto *Create*, geram exceções se o argumento *FileId* contém um UFID inválido ou se o usuário não tem direitos de acesso suficientes. Essas exceções foram omitidas da definição por motivos de clareza.

<i>Read</i> (FileId, <i>i</i> , <i>n</i>) → <i>Data</i> — gera <i>BadPosition</i>	Se $1 \leq i \leq \text{Length}(\text{File})$: lê uma sequência de até <i>n</i> elementos de um arquivo, começando no elemento <i>i</i> , e a retorna em <i>Data</i> . Gera uma exceção se o valor <i>i</i> é inválido.
<i>Write</i> (FileId, <i>i</i> , <i>Data</i>) — gera <i>BadPosition</i>	Se $1 \leq i \leq \text{Length}(\text{File})+1$: grava uma sequência de <i>Data</i> em um arquivo, começando no elemento <i>i</i> , ampliando o arquivo, se necessário. Gera uma exceção se o valor <i>i</i> é inválido.
<i>create</i> () → <i>FileId</i>	Cria um novo arquivo de tamanho zero e gera um UFID para ele.
<i>Delete</i> (FileId)	Remove o arquivo.
<i>GetAttributes</i> (FileId) → <i>Attr</i>	Retorna os atributos do arquivo.
<i>SetAttributes</i> (FileId, <i>Attr</i>)	Configura os atributos do arquivo (somente os atributos que não estão sombreados na Figura 12.3).

Figura 12.6 Operações do serviço de arquivos plano.

As operações mais importantes são as de leitura e escrita. Tanto a operação *Read* (leitura) como a operação *Write* (escrita) necessitam de um parâmetro *i* para especificar uma posição no arquivo. A operação *Read* copia para *Data* a sequência de *n* elementos de dados a partir do elemento *i* do arquivo especificado. A operação *Write* escreve, a partir do elemento *i*, a sequência de elementos de dados em *Data* para o arquivo especificado, substituindo o conteúdo anterior do arquivo na posição correspondente e, se necessário, aumentando o arquivo.

Create cria um novo arquivo vazio e retorna o UFID gerado. *Delete* remove o arquivo especificado.

GetAttributes e *SetAttributes* permitem que os clientes acessem o registro de atributos. Normalmente, *GetAttributes* está disponível para qualquer cliente que tenha permissão para ler o arquivo. Normalmente, o acesso à operação *SetAttributes* é restrito ao serviço de diretório que dá acesso ao arquivo. Os valores das partes relativas ao tamanho e ao carimbo de tempo do registro de atributos não são afetados por *SetAttributes*; eles são mantidos separadamente pelo próprio serviço de arquivos plano.

Comparação com o UNIX: nossa interface e as primitivas do sistema de arquivos UNIX são funcionalmente equivalentes. Basta construir um módulo cliente que simule as chamadas de sistema UNIX em termos de nosso serviço de arquivos plano e das operações do serviço de diretório descritas na próxima seção.

Em comparação com a interface UNIX, nosso serviço de arquivos plano não tem as operações *open* e *close* – os arquivos podem ser acessados imediatamente citando-se o UFID apropriado. Em nossa interface, as funções *Read* e *Write* incluem um parâmetro especificando um ponto de partida dentro do arquivo para cada transferência, enquanto as operações equivalentes do UNIX não incluem. No UNIX, cada operação *read* ou *write* começa na posição corrente do ponteiro de leitura e escrita, e esse ponteiro avança de acordo com o número de bytes transferidos após cada operação *read* ou *write*. É fornecida uma operação *seek* para permitir que o ponteiro de leitura e escrita seja posicionado explicitamente.

A interface para nosso serviço de arquivos plano difere da interface de sistema de arquivos do UNIX principalmente por motivos de tolerância a falhas:

Operações que podem ser repetidas: com exceção de *Create*, as operações são *idempotentes*, permitindo o uso da semântica RPC *pelo menos uma vez* – os clientes podem repetir chamadas para as quais não receberam resposta. A execução repetida de *create* produz um novo arquivo diferente para cada chamada.

Servidores sem estado (stateless): a interface é conveniente para a implementação por servidores sem estado. Um servidor sem estado pode ser reiniciado após uma

falha e retomar a operação sem necessidade dos clientes, ou do servidor, para restaurar qualquer estado.

As operações de arquivo do UNIX não são idempotentes, nem consistentes, com o requisito de uma implementação sem estado. Quando um arquivo é aberto, um ponteiro de leitura e escrita é gerado pelo sistema de arquivos do UNIX, e ele é mantido, junto aos resultados das verificações de controle de acesso até que o arquivo seja fechado. As operações *read* e *write* do UNIX não são idempotentes; se uma operação for repetida acidentalmente, o avanço automático do ponteiro de leitura e escrita resultará no acesso a uma parte diferente do arquivo na operação repetida. O ponteiro de leitura e escrita é uma variável (oculta) de estado relacionado ao cliente. Para imitá-la em um servidor de arquivos, seriam necessárias as operações *open* e *close*, e o valor do ponteiro de leitura e escrita teria de ser mantido pelo servidor enquanto o arquivo relevante estivesse aberto. Eliminando o ponteiro de leitura e escrita, eliminamos a parte da necessidade do servidor de arquivos de manter informações de estado em nome de clientes específicos.

Controle de acesso • No sistema de arquivos do UNIX, os direitos de acesso do usuário são verificados com relação ao *modo* de acesso (leitura ou escrita) solicitado na chamada *open* (a Figura 12.4 mostra a API do sistema de arquivos do UNIX), e o arquivo é aberto somente se o usuário tiver os direitos necessários. A identidade de usuário (UID) utilizada na verificação dos direitos de acesso é o resultado do *login* anteriormente autenticado do usuário e não pode ser falsificada em implementações não distribuídas. Os direitos de acesso resultantes são mantidos até que o arquivo seja fechado, e mais nenhuma verificação é exigida quando são solicitadas operações subsequentes no mesmo arquivo.

Nas implementações distribuídas, as verificações de direitos de acesso não precisam ser realizadas no servidor, pois, sob outros aspectos, a interface RPC do servidor é um ponto desprotegido de acesso aos arquivos. Uma identidade de usuário precisa ser passada com as requisições, e o servidor é vulnerável a identidades falsificadas. Além disso, se os resultados de uma verificação de direitos de acesso fossem mantidos no servidor e usados para acessos futuros, o servidor não seria mais sem estado. Podem ser adotadas duas estratégias alternativas para este último problema:

- É feita uma verificação de acesso quando um nome de arquivo é convertido em um UFID, e os resultados são codificados na forma de uma capacidade (veja a Seção 11.2.4), a qual é retornada para o cliente, para envio com as requisições subsequentes.
- Uma identidade de usuário é enviada com cada requisição de cliente e as verificações de acesso são realizadas pelo servidor para cada operação de arquivo.

Os dois métodos permitem implementação de servidor sem estado e ambos têm sido usados em sistemas de arquivos distribuídos. O segundo é mais comum; ele é usado no NFS e no AFS. Nenhuma dessas estratégias resolve o problema de segurança relativo às identidades de usuário falsificadas. Vimos, no Capítulo 11, que isso pode ser tratado com o uso de assinaturas digitais. O Kerberos é um esquema de autenticação eficaz que tem sido aplicado no NFS e no AFS.

Em nosso modelo abstrato, não fazemos nenhuma suposição sobre o método pelo qual o controle de acesso é implementado. A identidade do usuário é passada como um parâmetro implícito e pode ser usada quando for necessário.

Interface do serviço de diretório • A Figura 12.7 contém uma definição da interface RPC para um serviço de diretório. O principal objetivo do serviço de diretório é fornecer a

<i>Lookup</i> (<i>Dir</i> , <i>Name</i>) → <i>FileId</i> — gera <i>NotFound</i>	Localiza o nome textual no diretório e retorna o UFID relevante. Se <i>Name</i> não estiver no diretório, gera uma exceção.
<i>AddName</i> (<i>Dir</i> , <i>Name</i> , <i>FileId</i>) — gera <i>NameDuplicate</i>	Se <i>Name</i> não estiver no diretório, adiciona (<i>Name</i> , <i>File</i>) no diretório e atualiza o registro de atributos do arquivo. Se <i>Name</i> já estiver no diretório, gera uma exceção.
<i>UnName</i> (<i>Dir</i> , <i>Name</i>) — gera <i>NotFound</i>	Se <i>Name</i> estiver no diretório, a entrada contendo <i>Name</i> é removida do diretório. Se <i>Name</i> não estiver no diretório, gera uma exceção.
<i>GetNames</i> (<i>Dir</i> , <i>Pattern</i>) → <i>NameSeq</i>	Retorna todos os nomes textuais presentes no diretório que correspondam à expressão regular <i>Pattern</i> .

Figura 12.7 Operações do serviço de diretório.

transformação de nomes textuais em UFIDs. Para isso, ele mantém arquivos de diretório contendo os mapeamentos entre nomes textuais de arquivos e UFIDs. Cada diretório é armazenado como um arquivo convencional com um UFID, de modo que o serviço de diretório é um cliente do serviço de arquivos.

Definimos apenas operações sobre diretórios individuais. Para cada operação, é exigido o UFID do arquivo que contém o diretório (no parâmetro *Dir*). A operação *Lookup* no serviço de diretório realiza a transformação *Nome* → *UFID*. Ela é um bloco básico para uso em outros serviços, ou no módulo cliente, para realizar transformações mais complexas, como a interpretação de nome hierárquica encontrada no UNIX. Como antes, as exceções causadas por direitos de acesso inadequados foram omitidas das definições.

Existem duas operações para alterar diretórios: *AddName* e *UnName*. *AddName* adiciona uma entrada em um diretório e incrementa o campo de contagem de referência no registro de atributos do arquivo.

UnName remove uma entrada de um diretório e decrementa a contagem de referência. Se a contagem de referência chegar a zero, o arquivo é removido. *GetNames* é fornecida para permitir que os clientes examinem o conteúdo dos diretórios e para implementar operações de correspondência de padrão em nomes de arquivo, como aquelas encontradas no *shell* do UNIX. Ela retorna todos os nomes (ou um subconjunto deles) armazenados em determinado diretório. Os nomes são selecionados pela correspondência de padrão contra uma expressão regular fornecida pelo cliente.

A existência da correspondência de padrão na operação *GetNames* permite que os usuários determinem os nomes de um ou mais arquivos fornecendo uma especificação incompleta dos caracteres presentes nos nomes. Uma expressão regular é uma especificação para uma classe de *strings*, na forma de uma expressão contendo uma combinação de *substrings* literais e símbolos denotando caracteres variáveis ou ocorrências repetidas de caracteres ou *substrings*.

Sistema de arquivos hierárquico • Um sistema de arquivos hierárquico, como aquele fornecido pelo UNIX, consiste em vários diretórios organizados em uma estrutura em árvore. Cada diretório contém os nomes dos arquivos e de outros diretórios que podem ser acessados a partir dele. Qualquer arquivo ou diretório pode ser referenciado usando-se um *nome de caminho* (*pathname*) — um nome composto de várias partes que representa um caminho através da árvore. A raiz tem um nome distinto e cada arquivo, ou (sub)dire-

tório, possui um nome definido pelo usuário e armazenado em um diretório. O esquema de atribuição de nomes de arquivo do UNIX não é rigorosamente hierárquico – os arquivos podem ter vários nomes (*alias*) que podem estar no mesmo diretório ou em diretórios diferentes. Isso é implementado por meio de uma operação *link*, a qual adiciona um novo nome para um arquivo em um diretório especificado.

Um sistema de atribuição de nomes de arquivos como o do UNIX pode ser implementado pelo módulo cliente por meio dos serviços de arquivos plano e de diretório já definidos. Uma hierarquia de diretórios estruturada em árvore é construída com arquivos nas folhas e diretórios nos outros nós da árvore. A raiz da árvore é um diretório com um UFID conhecido. É possível atribuir vários nomes a um arquivo por meio da operação *AddName* e com o campo de contagem de referência no registro de atributos.

No módulo cliente, pode ser fornecida uma função que obtenha o UFID de um arquivo a partir de seu nome de caminho. A função interpreta o nome de caminho a partir da raiz, usando *Lookup* para obter o UFID de cada parte presente no caminho.

Em um serviço de diretório hierárquico, os atributos associados aos arquivos devem incluir um campo de tipo que faça a distinção entre arquivos normais e de diretórios. Isso é usado ao se seguir um caminho para garantir que cada parte do nome, exceto a última, refira-se a um diretório.

Grupos de arquivos • Um *grupo de arquivos* é um conjunto de arquivos localizado em determinado servidor. Um servidor pode conter vários grupos de arquivos e os grupos podem ser movidos entre os servidores, mas um arquivo não pode mudar do grupo a que pertence. Uma construção semelhante (*filesystem*) é usada no UNIX e na maioria dos outros sistemas operacionais. Os grupos de arquivos foram introduzidos originalmente para suportar recursos para mover conjuntos de arquivos armazenados em mídia removível entre computadores. Em um serviço de arquivos distribuído, os grupos de arquivos suportam a alocação de arquivos em unidades lógicas maiores e permitem que o serviço seja implementado com arquivos armazenados em vários servidores. Em um sistema de arquivos distribuído que suporta grupos de arquivos, a representação de UFIDs inclui um componente identificador de grupo de arquivos, permitindo que o módulo cliente assuma a responsabilidade por enviar as requisições para o servidor que contém o grupo de arquivos em questão.

Os identificadores de grupo de arquivos devem ser únicos em todo um sistema distribuído. Como os grupos de arquivos podem ser movidos, os sistemas de arquivos inicialmente separados podem ser combinados para formar um só sistema de arquivo, e a única maneira de garantir que os identificadores de grupo de arquivos sejam sempre diferentes em determinado sistema de arquivo é gerá-los com um algoritmo que garanta a exclusividade global. Por exemplo, quando um novo grupo de arquivos é criado, um identificador exclusivo pode ser gerado pela concatenação do endereço IP de 32 bits do computador que está criando o novo grupo, com um inteiro de 16 bits derivado da data, produzindo um inteiro único de 48 bits:

	32 bits	16 bits
identificador de grupo de arquivos:	endereço IP	data

Note que o endereço IP não pode ser usado para localizar o grupo de arquivos, pois ele pode ser movido para outro servidor. Em vez disso, o serviço de arquivo deve manter um mapeamento entre os identificadores de grupo e os servidores.

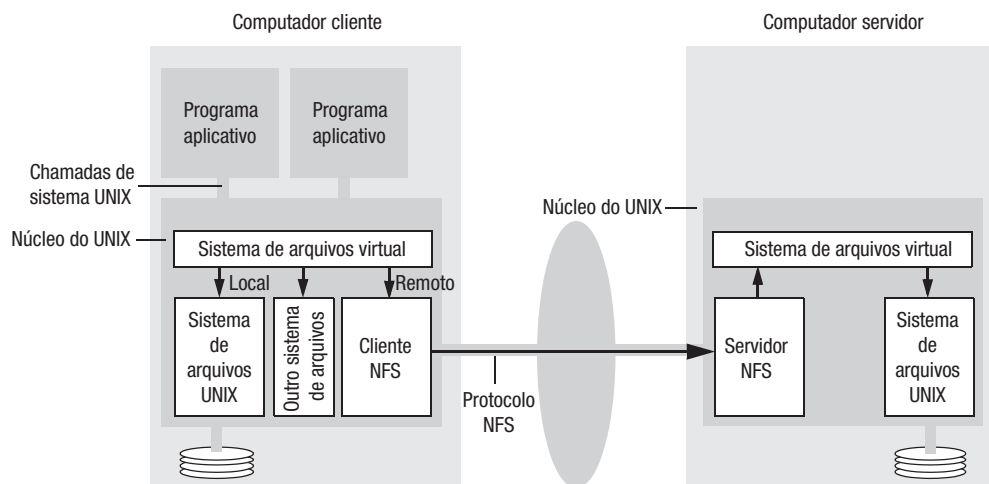


Figura 12.8 Arquitetura do NFS.

12.3 Estudo de caso: Sun Network File System

A Figura 12.8 mostra a arquitetura do NFS. Ela segue o modelo abstrato definido na seção anterior. Todas as implementações de NFS suportam o protocolo NFS – um conjunto de chamadas de procedimentos remotos que fornece o meio para os clientes efetuarem operações em um meio de armazenamento de arquivos remoto. O protocolo NFS é independente do sistema operacional, mas foi originalmente desenvolvido para uso em redes UNIX, e vamos descrever a implementação UNIX do protocolo NFS (versão 3).

O módulo *servidor NFS* reside no núcleo de cada computador que atua como servidor NFS. As requisições que se referem a arquivos de um sistema de arquivos remoto são transformadas em operações do protocolo NFS pelo módulo cliente e depois passadas para o módulo servidor NFS no computador que contém o sistema de arquivos em questão.

Os módulos cliente e servidor NFS se comunicam usando chamadas de procedimentos remotos. O sistema RPC da Sun, descrito na Seção 5.3.3, foi desenvolvido para ser usado no NFS. Ele pode ser configurado para usar UDP ou TCP, e o protocolo NFS é compatível com ambos. É incluído um serviço mapeador de porta (*portmapper*) para permitir que os clientes se associem, por meio de um nome, aos serviços de determinado computador. A interface RPC do servidor NFS é aberta: qualquer processo pode enviar requisições para um servidor NFS; caso as requisições sejam válidas e incluam credenciais de usuários autorizados, elas serão atendidas. Como um recurso de segurança opcional, pode-se exigir o envio de credenciais de usuário assinadas, assim como a criptografia dos dados para se obter privacidade e integridade.

Sistema de arquivos virtual • A Figura 12.8 torna claro que o NFS fornece transparência de acesso: os programas de usuário podem executar operações de arquivo, em arquivos locais ou remotos, sem distinção. Podem estar presentes outros sistemas de arquivos distribuídos que suportem chamadas de sistema UNIX e, se assim for, eles seriam integrados da mesma maneira.

A integração é obtida por meio de um módulo de sistema de arquivos virtual (VFS, Virtual File System), o qual foi adicionado no núcleo do UNIX para fazer a distinção entre arquivos locais e remotos, e para transformar os identificadores de arquivo usados

pelo NFS em identificadores internos normalmente usados no UNIX e em outros sistemas de arquivos. Além disso, o VFS controla os sistemas de arquivos que estão correntemente disponíveis tanto de forma local como remota e passa cada requisição para o módulo apropriado (o sistema de arquivos UNIX, o módulo de cliente NFS ou o módulo de serviço de outro sistema de arquivos).

Os identificadores de arquivo usados no NFS são chamados de *manipuladores de arquivo*. Um manipulador de arquivo não é visível para os clientes e contém as informações de que o servidor precisa para distinguir um arquivo individual. Nas implementações UNIX do NFS, o manipulador de arquivo é derivado do *i-node* do arquivo, adicionando-se dois campos extras, como dado a seguir (o *i-node* de um arquivo UNIX é um número que serve para identificar e localizar o arquivo dentro do sistema de arquivos no qual ele está armazenado):

<i>Manipulador de arquivo:</i>	Identificador do sistema de arquivos	<i>i-node</i> do arquivo	número de geração do <i>i-node</i>
--------------------------------	--------------------------------------	--------------------------	------------------------------------

O NFS adota a montagem de sistemas de arquivos* (*filesystem*) do UNIX como unidade de agrupamento de arquivos, definida na seção anterior. O campo *identificador de filesystem* é um número exclusivo alocado para cada sistema de arquivos (*filesystem*) quando ele é criado (e, na implementação UNIX, é armazenado no superbloco do sistema de arquivos). O número de geração do *i-node* é necessário porque no sistema de arquivos UNIX convencional os *i-nodes* são reutilizados após um arquivo ser removido. Nas extensões VFS do sistema de arquivos UNIX, um número de geração é armazenado com cada arquivo e é incrementado sempre que um *i-node* é reutilizado (por exemplo, em uma chamada de sistema *creat*). O cliente obtém o primeiro manipulador de arquivo para um sistema de arquivo remoto ao montá-lo. Os manipuladores de arquivo são passados do servidor para o cliente nos resultados das operações *lookup*, *create* e *mkdir* (veja a Figura 12.9) e do cliente para o servidor como um dos argumentos que as requisições contêm.

A camada de sistema de arquivos virtual tem uma estrutura VFS para cada sistema de arquivos (*filesystem*) montado e um *v-node* por arquivo aberto. Uma estrutura VFS relaciona um sistema de arquivo remoto com o diretório local em que ele é montado. O *v-node* contém uma indicação se um arquivo é local ou remoto. Se o arquivo é local, o *v-node* contém uma referência para o índice do arquivo local (um *i-node* em uma implementação UNIX). Se o arquivo é remoto, ele contém o manipulador de arquivo do arquivo remoto.

Integração com o cliente • O módulo cliente NFS desempenha o papel descrito para o módulo cliente em nosso modelo básico, fornecendo uma interface conveniente para os programas aplicativos. Porém, ao contrário do nosso modelo de módulo cliente, ele simula precisamente a semântica das primitivas padrão do sistema de arquivos UNIX e é integrado com o núcleo UNIX. As principais razões para que ele seja integrado, e não fornecido como uma biblioteca a ser ligada nos processos clientes, são para permitir que:

- os programas de usuário possam acessar arquivos por meio de chamadas de sistema UNIX sem serem ligados e compilados novamente;
- um único módulo cliente atenda a todos os processos em nível de usuário, com uma cache compartilhada dos blocos usados recentemente (descrito a seguir);

* N. de R.T.: Em inglês, há uma distinção entre os termos *filesystem* (uma única palavra) e *file system* (duas palavras). Por *filesystem*, entende-se o conjunto de arquivos e diretórios mantidos em uma partição de um dispositivo de armazenamento; *file system* refere-se ao módulo de *software* do sistema operacional que provê acesso aos arquivos. Corriqueiramente, em português, o termo *sistema de arquivos* engloba as duas funcionalidades e é empregado para referenciar ambos. Quando essa distinção for necessária, usaremos os termos em inglês entre parênteses.

<i>lookup(dirfh, name) → fh, attr</i>	Retorna o manipulador de arquivo e os atributos do arquivo <i>name</i> no diretório <i>dirfh</i> .
<i>create(dirfh, name, attr) → newfh, attr</i>	Cria um arquivo <i>name</i> no diretório <i>dirfh</i> com atributos <i>attr</i> e retorna o novo manipulador de arquivo e os seus atributos.
<i>remove(dirfh, name) → status</i>	Remove o arquivo <i>name</i> do diretório <i>dirfh</i> .
<i>getattr(fh) → attr</i>	Retorna os atributos do arquivo <i>fh</i> . (Semelhante à chamada de sistema <i>stat</i> do UNIX.)
<i>setattr(fh, attr) → attr</i>	Configura os atributos (modo, ID de usuário, ID de grupo, tamanho, horário de acesso e de modificação de um arquivo). Configurar o tamanho como zero trunca o arquivo.
<i>read(fh, offset, count) → attr, data</i>	Retorna até <i>count</i> bytes de dados de um arquivo, a partir de <i>offset</i> . Além disso, retorna os últimos atributos do arquivo.
<i>write(fh, offset, count, data) → attr</i>	Grava <i>count</i> bytes de dados em um arquivo, a partir de <i>offset</i> . Retorna os atributos do arquivo após a gravação terminar.
<i>rename(dirfh, name, todirfh, toname) → status</i>	Muda o nome do arquivo <i>name</i> no diretório <i>dirfh</i> para <i>toname</i> no diretório para <i>dirfh</i> .
<i>link(newdirfh, newname, fh) → status</i>	Cria uma entrada <i>newname</i> no diretório <i>newdirfh</i> referindo-se ao arquivo ou diretório <i>fh</i> .
<i>symlink(newdirfh, newname, string) → status</i>	Cria uma entrada <i>newname</i> no diretório <i>newdirfh</i> de tipo <i>vínculo simbólico</i> (<i>link</i> simbólico) com o valor <i>string</i> . O servidor não interpreta o <i>string</i> , mas produz um arquivo de vínculo simbólico para contê-la.
<i>readlink(fh) → string</i>	Retorna o <i>string</i> associado ao arquivo de vínculo simbólico identificado por <i>fh</i> .
<i>mkdir(dirfh, name, attr) → newfh, attr</i>	Cria um novo diretório <i>name</i> com atributos <i>attr</i> e retorna seu manipulador de arquivo e atributos.
<i>rmdir(dirfh, name) → status</i>	Remove o diretório vazio <i>name</i> do diretório pai <i>dirfh</i> . Falha se o diretório não estiver vazio.
<i>readdir(dirfh, cookie, count) → entries</i>	Retorna até <i>count</i> bytes de entradas do diretório <i>dirfh</i> . Cada entrada contém um nome de arquivo, um manipulador de arquivo e um ponteiro para a próxima entrada de diretório, chamado de <i>cookie</i> . O <i>cookie</i> é usado nas chamadas de <i>readdir</i> subsequentes para começar a ler a partir da entrada seguinte. Se o valor de <i>cookie</i> for zero, lê a partir da primeira entrada no diretório.
<i>statfs(fh) → fsstats</i>	Retorna informações do sistema de arquivos (como o tamanho do bloco, número de blocos livres, etc.) que contém um arquivo <i>fh</i> .

Figura 12.9 Operações do servidor NFS (protocolo NFS versão 3, simplificado).

- a chave de criptografia usada para autenticar as IDs de usuário passadas para o servidor (veja a seguir) possa ser mantida no núcleo, impedindo a personificação por parte de clientes em nível de usuário.

O módulo cliente NFS coopera com o sistema de arquivos virtual em cada máquina cliente. Ele funciona de maneira semelhante ao sistema de arquivos convencional do UNIX, transferindo blocos de arquivos para o servidor (e dele) e, quando possível, colocando os blocos em cache na memória local. Ele compartilha a mesma cache usada pelo sistema de entrada e saída local. Porém, como vários clientes, em diferentes máquinas, podem acessar simultaneamente o mesmo arquivo remoto, surge um problema de consistência de cache.

Controle de acesso e autenticação • Ao contrário do sistema de arquivos UNIX convencional, o servidor NFS é sem estado e não mantém arquivos abertos no nome de seus clientes.

Portanto, a cada requisição, o servidor deve verificar novamente a identidade do usuário nos atributos de permissão de acesso do arquivo, para ver se o usuário pode acessar o arquivo da maneira solicitada. O protocolo RPC Sun exige que os clientes enviem informações de autenticação de usuário (por exemplo, a ID de usuário e a ID de convencionais – em 16 bits – do UNIX) com cada requisição, e isso é verificado em relação à permissão de acesso presente nos atributos do arquivo. Esses parâmetros adicionais não aparecem em nossa visão geral do protocolo NFS da Figura 12.9; eles são fornecidos automaticamente pelo sistema RPC.

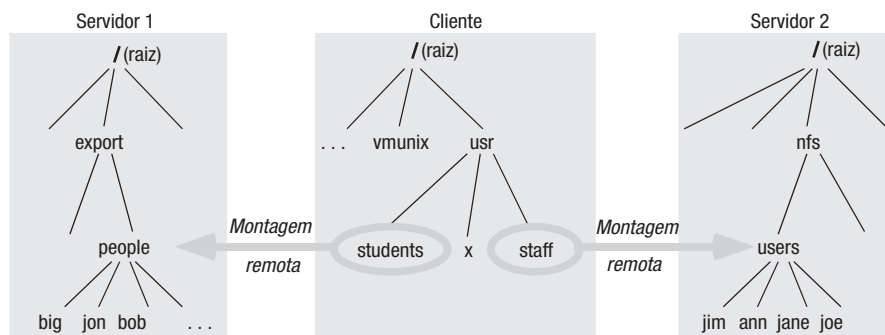
Em sua forma mais simples, existe uma brecha de segurança nesse mecanismo de controle de acesso. Um servidor NFS fornece uma interface RPC convencional em uma porta bem conhecida em cada computador, e qualquer processo pode se comportar como cliente, enviando requisições para o servidor para acessar ou atualizar um arquivo. O cliente pode modificar as chamadas de RPC para incluir a ID de qualquer usuário, personificando-o sem seu conhecimento ou permissão. Essa brecha de segurança foi fechada pelo uso de uma opção do protocolo RPC para a criptografia DES das informações de autenticação do usuário. Mais recentemente, o Kerberos foi integrado com o Sun NFS para proporcionar uma solução mais forte e abrangente para os problemas da autenticação de usuário e da segurança, e descreveremos isso a seguir.

Interface do servidor NFS • Uma representação simplificada da interface RPC fornecida pelos servidores NFS versão 3 (definidos no RFC 1813 [Callaghan *et al.* 1995]) aparece na Figura 12.9. As operações de acesso a arquivo do NFS, *read*, *write*, *getattr* e *setattr* são quase idênticas às operações *Read*, *Write*, *GetAttributes* e *SetAttributes* definidas para nosso modelo de serviço de arquivo simples (Figura 12.6). A operação *lookup* e a maior parte das outras operações de diretório, definidas na Figura 12.9, são semelhantes às aquelas de nosso modelo de serviço de diretório (Figura 12.7).

As operações de arquivo e diretório são integradas em um único serviço; a criação e a inserção de nomes de arquivo em diretórios são realizadas por uma única operação *create*, que recebe como argumentos o nome textual do novo arquivo e o manipulador do arquivo do diretório de destino. As outras operações NFS sobre diretórios são *create*, *remove*, *rename*, *link*, *symlink*, *readlink*, *mkdir*, *rmdir*, *readdir* e *statfs*. Elas são semelhantes às suas correlatas do UNIX, com exceção de *readdir*, que fornece um método independente de representação para ler o conteúdo de diretórios, e *statfs*, que fornece as informações de status em sistemas de arquivos remotos.

Serviço de montagem • A montagem de subárvores de sistemas de arquivos (*filesystems*) remotos feita por clientes é suportada por um processo separado, o *serviço de montagem*, executado em nível de usuário em cada computador servidor NFS. Em cada servidor existe um arquivo com um nome conhecido (*/etc/exports*), contendo os nomes dos sistemas de arquivos locais (*local filesystems*) que estão disponíveis para montagem remota. Uma lista de acesso é associada a cada nome de sistema de arquivos (*filesystem*), indicando quais máquinas podem montá-los.

Os clientes usam uma versão modificada do comando *mount* do UNIX para solicitar a montagem de um sistema de arquivos remoto, especificando o nome de computador remoto, o nome de caminho de um diretório no sistema de arquivos remoto e o nome local com o qual ele será montado. O diretório remoto pode ser qualquer subárvore do sistema de arquivo remoto solicitado, permitindo aos clientes montarem qualquer parte sua. O comando *mount* modificado se comunica com o processo do serviço de montagem no computador remoto, usando um *protocolo de montagem*. Trata-se de um protocolo RPC que inclui uma operação que recebe um nome de caminho de diretório e retorna o manipulador do arquivo do diretório especificado, caso o cliente tenha permissão de acesso para o sistema de arqui-



Nota: O sistema de arquivos montado em `/usr/students` no cliente é, na verdade, a subárvore localizada em `/export/people` no Servidor 1; o sistema de arquivos montado em `/usr/staff` no cliente é, na verdade, a subárvore localizada em `/nfs/users` no Servidor 2.

Figura 12.10 Sistemas de arquivos local e remoto acessíveis em um cliente NFS.

vos relevante. A localização (endereço IP e número de porta) do servidor e do manipulador de arquivo do diretório remoto são passados na camada VFS e no cliente NFS.

A Figura 12.10 ilustra um *Cliente* com dois sistemas de arquivos montados de forma remota. Os nós *people* e *users* nos sistemas de arquivos de *Servidor 1* e *Servidor 2* são montados sobre os nós *students* e *staff* no sistema de arquivos local do *Cliente*. O significado disso é que os programas executados no *Cliente* podem acessar arquivos em *Servidor 1* e em *Servidor 2*, usando nomes de caminho como `/usr/students/jon` e `/usr/staff/ann`.

Os sistemas de arquivos remotos podem ser *montados incondicionalmente* ou *condicionalmente* em um computador cliente. Quando um processo em nível de usuário acessa um arquivo em um sistema de arquivos montado incondicionalmente, o processo é suspenso até que a requisição possa ser concluída e, se o computador remoto estiver indisponível por qualquer razão, o módulo cliente NFS continuará a fazer novas tentativas até ser atendido. Assim, no caso de uma falha do servidor, os processos em nível de usuário são suspensos até que o servidor seja reiniciado e, depois, continuam normalmente. No entanto, se o sistema de arquivos relevante for montado condicionalmente, o módulo cliente NFS retornará uma indicação de erro para os processos em nível de usuário, após um pequeno número de novas tentativas. Então, os programas construídos corretamente detectarão a falha e executarão ações apropriadas de recuperação ou de geração de relatórios. Entretanto, muitos utilitários e aplicativos UNIX não testam a existência de falhas em operações de acesso a arquivo e se comportam de maneiras imprevisíveis no caso de falha de um sistema de arquivos montado condicionalmente. Por isso, muitas instalações utilizam exclusivamente a montagem incondicional, com a consequência de que os programas são incapazes de se recuperar normalmente quando um servidor NFS fica indisponível por um período de tempo significativo.

Tradução de nomes de caminho • Os sistemas de arquivos UNIX traduzem *nomes de caminho* de arquivos composto por muitas partes em referências a *i-nodes*, em um processo passo a passo, quando são usadas as chamadas de sistema *open*, *creat* ou *stat*. No NFS, os nomes de caminho não podem ser traduzidos em um servidor, pois um nome pode cruzar um ponto de montagem no cliente – diretórios contendo várias partes em nome podem residir em sistemas de arquivos de diferentes servidores. Portanto, os nomes de caminho são analisados e sua tradução é realizada de maneira iterativa pelo cliente. Cada parte de um nome que se refere a um diretório montado de forma remota é transformada em um manipulador de arquivo usando uma requisição de *lookup* separada para o servidor remoto.

A operação *lookup* procura uma única parte de um nome de caminho em determinado diretório e retorna o manipulador de arquivo e os atributos correspondentes. O manipulador de arquivo retornado em um passo anterior é usado como parâmetro no próximo passo da operação *lookup*. Como os manipuladores de arquivo são opacos (não interpretados) para o código do cliente NFS, o sistema de arquivos virtual é responsável por traduzir o manipulador de arquivo em um diretório local, ou remoto, e por realizar o procedimento indireto necessário quando fizer referência a um ponto de montagem local. O armazenamento em cache dos resultados de cada passo diminui a aparente ineficiência desse processo, tirando proveito do caráter local da referência a arquivos e diretórios; isto é, normalmente, os usuários e os programas acessam arquivos em apenas um diretório ou em um pequeno número de diretórios.

Montagem automática (*automounter*) • A montagem automática (*automounter*) foi adicionada na implementação UNIX do NFS para montar dinamicamente um diretório remoto quando um ponto de montagem “vazio” fosse referenciado por um cliente. A implementação original da montagem automática era executada como um processo UNIX em nível de usuário em cada computador cliente. Versões posteriores (chamadas de *autofs*) foram implementadas no núcleo do Solaris e do Linux. Descreveremos aqui a versão original.

O *automounter* mantém uma tabela de pontos de montagem (nomes de caminho), com uma referência, para cada um deles, de um ou mais servidores NFS. Ele se comporta como um servidor NFS local na máquina cliente. Quando o módulo cliente NFS tenta solucionar um nome de caminho que inclui um desses pontos de montagem, ele faz uma requisição de *lookup()* para o *automounter* local, que localiza em sua tabela o sistema de arquivos exigido e envia uma requisição de *sondagem (probe)* para cada servidor listado. O sistema de arquivos do primeiro servidor a responder é, então, montado no cliente, usando o serviço de montagem normal. O sistema de arquivos montado é associado ao ponto de montagem usando um vínculo simbólico (*symbolic link*), de modo que os acessos a ele não resultarão em mais requisições para o *automounter*. Então, o acesso ao arquivo prossegue normalmente, sem mais referências ao *automounter*, a não ser que não existam referências a esse vínculo simbólico por vários minutos. Neste caso, o *automounter* desmonta o sistema de arquivos remoto.

As implementações posteriores, no núcleo, substituíram os vínculos simbólicos por montagens reais, evitando alguns problemas que surgiam em aplicativos que armazenavam em cache os nomes de caminho temporários usados pelo *automounter* em nível de usuário [Callaghan 1999].

Uma forma simples de replicação, somente em leitura, pode ser obtida por um conjunto de vários servidores contendo cópias idênticas de um sistema de arquivos, ou de uma subárvore de arquivos, com uma única entrada na tabela do *automounter*. Isso é útil para sistemas de arquivos muito utilizados que mudam pouco, como os binários de sistema do UNIX. Por exemplo, cópias do diretório */usr/lib* e de sua subárvore poderiam ser mantidas em mais de um servidor. Na primeira ocasião em que um arquivo de */usr/lib* fosse aberto em um cliente, todos os servidores receberiam as mensagens de *probe* e o primeiro a responder seria montado no cliente. Isso proporciona um grau limitado de tolerância a falhas e balanceamento de carga, pois o primeiro servidor a responder será um que não falhou e é provavelmente um que não esteja muito ocupado com o atendimento de outras requisições.

Uso de cache no servidor • O uso de cache, tanto no cliente como no servidor, é um recurso indispensável para obter um desempenho adequado nas implementações do NFS.

Nos sistemas UNIX convencionais, blocos de arquivo, diretórios e atributos de arquivo que foram lidos do disco são mantidos em *cache* na memória principal até que o

espaço da cache seja exigido para outros blocos. Se um processo faz uma requisição de leitura ou escrita de um bloco que já está na cache, ele pode ser atendido sem outro acesso ao disco. A *leitura antecipada* (*read-ahead*) adianta os acessos de leitura e busca os blocos seguintes àqueles lidos mais recentemente, e a *escrita postergada* (*delayed-write*) otimiza as operações de escrita: quando um bloco tiver sido alterado (por uma requisição de escrita), seu novo conteúdo será gravado no disco somente quando o espaço ocupado por esse bloco na cache for exigido por outro. Para evitar a perda de dados no caso de falha do sistema, a operação *sync* do UNIX passa para o disco, a cada 30 segundos, os blocos alterados. Essas técnicas de uso de cache funcionam em um ambiente UNIX convencional porque todas as requisições de leitura e escritas feitas pelos processos em nível de usuário passam por uma única cache, que é implementada no espaço do núcleo do UNIX. A cache é sempre mantida atualizada e os acessos aos arquivos devem sempre passar por ela.

Os servidores NFS usam cache exatamente como ela é empregada para outros acessos a arquivos. O uso da cache do servidor para conter blocos de disco lidos recentemente não acarreta nenhum problema de consistência, mas quando um servidor realiza operações de escritas, medidas extras são necessárias para garantir que os clientes possam ter confiança de que os resultados das operações de escrita sejam persistentes, mesmo quando ocorre uma falha do servidor. Na versão 3 do protocolo NFS, a operação *write* oferece duas opções (não mostradas na Figura 12.9):

1. Nas operações *write*, os dados recebidos dos clientes são armazenados na cache no servidor e gravados no disco, antes que uma resposta seja enviada para o cliente. Isso é chamado de uso de cache com *escrita direta* (*write-through*). O cliente pode ter certeza de que os dados estão armazenados de modo persistente, assim que a resposta for recebida.
2. Nas operações *write*, os dados são armazenados apenas na cache de memória. Eles serão gravados no disco quando uma operação de confirmação (*commit*) for recebida para o arquivo relevante. O cliente tem certeza de que os dados estão armazenados de modo persistente somente quando a resposta da operação *commit* for recebida. Os clientes NFS padrão usam esse modo de operação, executando uma operação *commit* quando um arquivo aberto para gravação for fechado.

Commit é uma operação adicional fornecida na versão 3 do protocolo NFS; ela foi incluída para superar o gargalo de desempenho causado pelo modo de operação *write-through* nos servidores que recebem grandes quantidades de operações *write*.

O requisito de *write-through* em sistemas de arquivos distribuídos é um caso de modos de falha independentes, discutidos no Capítulo 1 – os clientes continuam a operar quando um servidor falha, e os programas aplicativos podem executar ações supondo que os resultados das escritas anteriores foram enviados para armazenamento em disco. É improvável que isso ocorra no caso de atualizações de arquivos locais, pois é quase certo que a falha de um sistema de arquivos local resulte na falha de todos os processos aplicativos em execução no mesmo computador.

Uso de cache no cliente • O módulo cliente NFS armazena em cache os resultados das operações *read*, *write*, *getattr*, *lookup* e *readdir*, para reduzir o número de requisições feitas aos servidores. O uso de cache no cliente implica a possibilidade de existirem diversas versões de arquivos, ou porções deles, em diferentes clientes, pois as escritas feitas por um cliente não resultam na atualização imediata em outros clientes das cópias do mesmo arquivo armazenados em cache. Em vez disso, os clientes são responsáveis por fazerem uma consulta sequencial no servidor para verificar se os dados armazenados em cache que eles contêm são atuais.

Um método baseado em carimbos de tempo é usado para validar blocos armazenados em cache antes de serem usados. Cada elemento de dados, ou de metadados, presente na cache é rotulado com dois carimbos de tempo (*timestamps*):

Tc é a hora em que a entrada da cache foi validada pela última vez.

Tm é a hora em que o bloco foi modificado pela última vez no servidor.

Uma entrada da cache é válida no tempo T , se $T - Tc$ for menor que um intervalo de atualização t , ou se o valor de Tm gravado no cliente corresponder ao valor Tm presente no servidor (isto é, os dados não foram modificados no servidor desde que a entrada na cache foi feita). Formalmente, a condição de validade é:

$$(T - Tc < t) \vee (Tm_{\text{cliente}} = Tm_{\text{servidor}})$$

A escolha de um valor para t é um compromisso entre consistência e eficiência. Um intervalo de atualização muito curto resultará em uma forte aproximação da consistência de cópia única, ao custo de uma carga relativamente pesada de chamadas para o servidor para verificar o valor de Tm_{servidor} . Nos clientes Sun Solaris, t é configurado de forma adaptativa para arquivos individuais, com um valor no intervalo de 3 a 30 segundos, dependendo da frequência das atualizações no arquivo. Para diretórios, o intervalo é de 30 a 60 segundos, refletindo o menor risco de atualizações concorrentes.

Existe um valor de Tm_{servidor} para todos os blocos de dados de um arquivo e outro para os atributos de arquivo. Como os clientes NFS não podem determinar se um arquivo está sendo compartilhado ou não, o procedimento de validação deve ser usado para todos os acessos ao arquivo. Uma verificação de validade é realizada quando uma entrada da cache é usada. A primeira metade da condição de validade pode ser avaliada sem nenhum acesso ao servidor. Se ela for verdadeira, então a segunda metade não precisará ser avaliada; se for falsa, o valor corrente de Tm_{servidor} será obtido (por meio de uma chamada de *getattr* no servidor) e comparado com o valor local Tm_{cliente} . Se eles forem iguais, então a entrada da cache será considerada válida, e o valor de Tc dessa entrada da cache será atualizado com o tempo corrente. Se eles diferirem, então os dados armazenados em cache foram atualizados no servidor e a entrada da cache será invalidada, resultando em uma requisição para o servidor fornecer os dados relevantes.

Várias medidas são usadas para reduzir o tráfego das chamadas de *getattr* para o servidor:

- Quando um novo valor de Tm_{servidor} é recebido em um cliente, ele é aplicado em todas as entradas de cache derivadas do arquivo relevante.
- Os valores de atributo correntes são enviados “a tiracolo” (*piggybacking*) com os resultados de cada operação em um arquivo e, se o valor de Tm_{servidor} tiver mudado, o cliente o utilizará para atualizar as entradas de cache relativas ao arquivo.
- O algoritmo adaptativo para configurar o intervalo de atualização t , mencionado anteriormente, reduz consideravelmente o tráfego para a maioria dos arquivos.

O procedimento de validação não garante o mesmo nível de consistência de arquivos que é fornecido nos sistemas UNIX convencionais, pois as atualizações recentes nem sempre são visíveis aos clientes que estão compartilhando um arquivo. Existem duas fontes de atraso temporais: o atraso após uma escrita e antes que os dados atualizados sejam removidos da cache do cliente, e a janela de 3 segundos para a validação da cache. Felizmente, a maioria dos aplicativos UNIX não depende da sincronização das atualizações de arquivo, e poucas dificuldades foram relatadas por causa dessa fonte.

As escritas são manipuladas de forma diferente. Quando um bloco armazenado em cache é modificado, ele é marcado como alterado (*dirty*) e programado para ser transferido para

o servidor de forma assíncrona. Os blocos modificados são transferidos quando o arquivo é fechado, quando ocorre uma operação *sync* no cliente, ou ainda, mais frequentemente, se processos *bio-daemon* estiverem em uso (veja a seguir). Isso não proporciona a mesma garantia de persistência que a cache do servidor, mas simula o comportamento das escritas locais.

Para implementar leitura antecipada (*read-ahead*) e escrita postergada (*write-delayed*), o cliente NFS precisa realizar algumas leituras e escritas de forma assíncrona. Isso é obtido nas implementações UNIX do NFS por meio da inclusão de um ou mais processos *bio-daemon* em cada cliente. (*Bio* significa entrada e saída de bloco [*block input-output*]; o termo *daemon* é frequentemente usado para referenciar processos em nível de usuário que executam tarefas de sistema.) A função dos processos *bio-daemon* é realizar operações de leitura antecipada e escrita postergada. Após cada requisição de leitura, o *bio-daemon* é notificado e, então, ele inicia a transferência, do servidor para a cache cliente, do bloco de arquivo seguinte ao que foi lido. No caso de escrita, o *bio-daemon* envia um bloco para o servidor sempre que o bloco tiver sido completamente escrito pelo cliente. Os blocos de diretório são enviados quando ocorre uma modificação.

Os processos *bio-daemon* melhoram o desempenho do sistema garantindo que o módulo cliente não fique bloqueado esperando pelo retorno de operações *read*, nem que operações *write* sejam efetivadas no servidor. Eles não são um requisito lógico obrigatório, pois na ausência de um mecanismo de leitura antecipada, uma operação *read* feita por um processo usuário, provocará uma requisição síncrona para o servidor; e os resultados das operações *write* serão transferidos para o servidor quando o arquivo for fechado ou quando o sistema de arquivos virtual no cliente executar uma operação *sync*.

Outras otimizações • O sistema de arquivos da Sun é baseado no *Fast File System* do BSD UNIX, que usa blocos de disco de 8 Kbytes, resultando em menos chamadas de sistema para acessar arquivos sequenciais do que os sistemas UNIX anteriores. Os datagramas UDP usados para a implementação da RPC Sun são ampliados para 9 Kbytes, permitindo que uma chamada de RPC, contendo um bloco inteiro como argumento, seja transferida em um único datagrama, minimizando o efeito da latência da rede na leitura sequencial de arquivos. No NFS versão 3, não há limite para o tamanho máximo de blocos de arquivo que podem ser manipulados em operações *read* e *write*; clientes e servidores podem negociar tamanhos maiores do que 8 Kbytes, se ambos puderem manipulá-los.

Conforme mencionado anteriormente, as informações de status do arquivo colocadas na cache dos clientes devem ser atualizadas pelo menos a cada três segundos para arquivos ativos. Para reduzir a consequente carga no servidor, resultante das requisições *getattr*, todas as operações que se referem a arquivos, ou diretórios, são consideradas requisições *getattr* implícitas, e os valores de atributo correntes são levados “a tiracolo”, junto aos outros resultados da operação.

Tornando o NFS seguro com o Kerberos • Na Seção 11.6.2, descrevemos o sistema de autenticação Kerberos, desenvolvido no MIT, que se transformou em um padrão para tornar servidores de intranet seguros contra acessos não autorizados e ataques de impostores. A segurança das implementações do NFS foi melhorada com o uso do esquema Kerberos para autenticar clientes. Nesta subseção, descreveremos a “kerberização” do NFS, conforme especificado pelos projetistas do Kerberos.

Na implementação padrão original do NFS, a identidade do usuário é incluída em cada requisição, na forma de um identificador numérico “em claro”. (Nas versões posteriores do NFS, o identificador podia ser cifrado.) O NFS não faz nada para verificar a autenticidade do identificador fornecido. Isso implica, por parte do NFS, um alto grau de confiança na integridade do computador cliente e em seu *software*; enquanto o objetivo

do Kerberos, e de outros sistemas de segurança baseados em autenticação, é reduzir a um mínimo a gama de componentes nos quais a confiança é pressuposta. Basicamente, quando o NFS é usado em um ambiente “kerberizado”, ele só deve aceitar requisições de clientes que possam mostrar que sua identidade foi autenticada pelo Kerberos.

Uma solução óbvia, considerada pelos desenvolvedores do Kerberos, foi alterar a natureza das credenciais exigidas pelo NFS para que fossem um tíquete e um autenticador completos do Kerberos. Contudo, como NFS é implementado como um servidor sem estado, cada requisição de acesso a arquivo é tratada isoladamente e, por consequência, os dados de autenticação devem ser incluídos em cada uma delas. Isso foi considerado inaceitavelmente dispendioso quanto ao tempo exigido para realizar os procedimentos de cifragem necessários e, além disso, acarretaria a adição da biblioteca de cliente Kerberos no núcleo de cada uma das estações de trabalho.

Em vez disso, foi adotada uma estratégia mista, na qual cada servidor NFS, ao montar os sistemas de arquivos de usuários e raiz, recebe os dados de autenticação completos do Kerberos. Os resultados dessa autenticação, incluindo o identificador numérico do usuário (*uid*) e o endereço do computador cliente, são mantidos pelo servidor com as informações de montagem para cada sistema de arquivos. (Embora o servidor NFS não mantenha o estado relacionado aos processos clientes individuais, ele mantém as montagens correntes em cada computador cliente).

Em cada requisição de acesso a arquivo, o servidor NFS verifica o identificador do usuário e o endereço do remetente, e só garante o acesso se eles corresponderem àqueles armazenados no servidor para o cliente relevante, no momento da montagem. Essa estratégia mista envolve apenas um custo adicional mínimo e é segura contra a maioria das formas de ataque, desde que apenas um usuário por vez possa se conectar em cada computador cliente. No MIT, o sistema é configurado de modo que esse seja o caso. As implementações de NFS recentes incluem autenticação do Kerberos como uma de várias opções de autenticação, e *sites* que também executam servidores Kerberos são aconselhados a utilizar essa opção.

Desempenho • Os primeiros resultados relatados por Sandberg [1987] mostraram que o uso de NFS, normalmente, não impunha uma penalidade sobre o desempenho em comparação com o acesso aos arquivos armazenados em discos locais. Ele identificou duas áreas problemáticas remanescentes:

- uso frequente da chamada de *getattr* para buscar carimbos de tempo nos servidores para validação da cache;
- desempenho relativamente deficiente da operação *write*, porque era usada escrita direta (*write-through*) no servidor.

Ele observou que as escritas são relativamente raras na carga de trabalho normal do UNIX (cerca de 5% de todas as chamadas para o servidor) e, portanto, o custo da escrita direta é tolerável, exceto quando arquivos grandes são gravados no servidor. A versão do NFS testada não incluía o mecanismo *commit* mencionado anteriormente, e isso resultou em uma melhoria substancial no desempenho de escrita nas versões atuais. Seus resultados também mostram que a operação *lookup* é responsável por quase 50% das chamadas ao servidor. Isso é uma consequência do método de tradução de nomes de caminho, passo a passo, necessário à semântica de atribuição de nomes de arquivos do UNIX.

Regularmente, medidas são realizadas pela Sun, e por outros implementadores de NFS, usando uma versão atualizada de um conjunto exaustivo de programas de *benchmark*, conhecido como LADDIS [Keith e Wittle 1993]. Resultados atuais e antigos estão disponíveis no endereço [www.spec.org]. Lá, está resumido o desempenho para implementações do servidor NFS de muitos fornecedores e de diferentes configurações de *hardware*. Implementações

com uma só CPU, baseadas em *hardware* de PC, mas com sistemas operacionais dedicados, atingem desempenho de mais de 12.000 operações por segundo no servidor, e configurações grandes, com multiprocessadores, vários discos e controladoras, atingirem cerca de até 300.000 operações por segundo no servidor. Esses valores indicam que o NFS oferece uma solução muito eficiente para as necessidades de armazenamento distribuído em intranets em diferentes tipos de uso, variando, por exemplo, desde uma carga de atividades em desenvolvimento, em um UNIX tradicional, executada por várias centenas de engenheiros de *software*, até um conjunto de servidores Web recuperando páginas de um único servidor NFS.

Resumo do NFS • O NFS segue de perto nosso modelo abstrato. O projeto resultante proporciona boa transparência de localização e de acesso, caso o serviço de montagem NFS seja usado corretamente para produzir espaços de nomes semelhantes em todos os clientes. O NFS suporta *hardware* e sistemas operacionais heterogêneos. A implementação do servidor NFS é sem estado (*stateless*), permitindo que clientes e servidores retomem a execução após uma falha, sem a necessidade de quaisquer procedimentos de recuperação. A migração de arquivos, ou de sistemas de arquivos, não é suportada, a não ser que seja feita por intervenção manual para reconfigurar diretivas de montagem, após a mudança de um sistema de arquivos para um novo local.

O desempenho do NFS melhora com o uso de cache para blocos de arquivo em cada computador cliente. Isso é importante para a obtenção de um desempenho satisfatório, mas resulta em certo desvio da semântica estrita de atualização de arquivo com cópia única do UNIX.

Os outros objetivos de projeto do NFS, e até que ponto eles foram atingidos, serão discutidos a seguir.

Transparência de acesso: o módulo cliente NFS fornece uma interface de programação de aplicativos para processos locais idêntica à interface do sistema operacional local. Assim, em um cliente UNIX, os acessos a arquivos remotos são realizados usando-se as chamadas de sistema UNIX normais. Nenhuma modificação é exigida nos programas existentes, para permitir que eles funcionem corretamente com arquivos remotos.

Transparência de localização: cada cliente estabelece um espaço de nomes de arquivo, adicionando diretórios montados em sistemas de arquivos remotos em seu espaço de nomes local. Os sistemas de arquivo precisam ser *exportados* pelo computador que os contém e *montados de forma remota* por um cliente, antes que possam ser acessados pelos processos em execução no cliente (veja a Figura 12.10). O ponto na hierarquia de nomes de um cliente em que um sistema de arquivos montado aparece é determinado pelo cliente; assim, o NFS não impõe um único espaço de nomes de arquivo em nível de rede – cada cliente vê um conjunto de sistemas de arquivos remotos de acordo com o ponto de montagem usado. Como consequência, sistemas de arquivos remotos podem ter diferentes nomes de caminho em diferentes clientes, mas um espaço de nomes uniforme pode ser estabelecido com tabelas de configuração apropriadas em cada cliente, atingindo o objetivo da transparência de localização.

Transparência de mobilidade: os sistemas de arquivos (no sentido do UNIX, isto é, subárvores de arquivos) podem ser movidos entre servidores, mas as tabelas de montagem em cada cliente devem ser atualizadas separadamente para permitir que os clientes acessem o sistema de arquivos em sua nova localização; portanto, a transparência da migração não é totalmente obtida pelo NFS.

Escalabilidade: os valores de desempenho publicados mostram que servidores NFS podem ser construídos para manipular cargas reais muito grandes de maneira eficiente e econômica. O desempenho de um único servidor pode ser aumentado

com a adição de processadores, discos e controladoras. Quando os limites desse processo são atingidos, servidores adicionais devem ser instalados, e os sistemas de arquivos devem ser realocados entre eles. A eficácia dessa estratégia é limitada pela existência de arquivos “concorridos” – aqueles que são acessados com tanta frequência que o servidor atinge um limite de desempenho. Quando a carga ultrapassa o desempenho máximo disponível com essa estratégia, uma solução melhor pode ser oferecida por um sistema de arquivos distribuído que suporte a replicação de arquivos (como o Coda, descrito no Capítulo 18), ou por um sistema de arquivos como o AFS, que reduz o tráfego do protocolo colocando arquivos inteiros na cache. Vamos discutir outras estratégias para atingir escalabilidade na Seção 12.5.

Replicação de arquivos: Meios de armazenamentos de arquivos somente para leitura (*read-only*) podem ser replicados em vários servidores NFS, mas o NFS não suporta replicação de arquivos com atualizações. O NIS (Network Information Service), da Sun, é um serviço separado, disponível para uso com o NFS, que suporta a replicação de bancos de dados simples organizados como pares chave-valor (por exemplo, os arquivos de sistema do UNIX */etc/passwd* e */etc/hosts*). Ele gerencia a distribuição de atualizações e acessos a arquivos replicados com base em um modelo de replicação mestre–escravo simples (também conhecido como modelo de *cópia primária*, melhor discutido no Capítulo 18), com possibilidade de replicação parcial ou total do banco de dados em cada *site*. O NIS fornece um repositório compartilhado de informações de sistema que muda com pouca frequência e não exige que as atualizações ocorram simultaneamente em todos os *sites*.

Heterogeneidade de hardware e de sistema operacional: o NFS foi implementado por quase todos os sistemas operacionais e plataformas de *hardware* disponíveis e é suportado por uma variedade de sistemas de arquivos.

Tolerância a falhas: as características sem estado e idempotente do protocolo de acesso NFS garantem que os modos de falha observados pelos clientes ao acessar arquivos remotos sejam semelhantes àqueles encontrados no acesso de arquivos locais. Quando um servidor falha, o serviço que ele fornece é suspenso até que o servidor seja reiniciado, mas uma vez que ele tenha sido reiniciado, os processos clientes em nível de usuário prosseguem a partir do ponto em que o serviço foi interrompido, sem conhecimento da falha (exceto no caso de acesso a sistemas de arquivos remotos *montados condicionalmente*). Na prática, a montagem incondicional é usada na maioria dos casos, e isso tende a impedir que os programas aplicativos tratem das falhas do servidor.

A falha de um computador cliente, ou de um processo usuário, não tem nenhum efeito sobre qualquer servidor que ele possa usar, pois os servidores não contêm nenhum estado para seus clientes.

Consistência: descrevemos o comportamento da atualização com alguns detalhes. Ele apresenta uma forte aproximação com a semântica de cópia única e atende às necessidades da maioria dos aplicativos; porém, o uso de compartilhamento de arquivos por meio do NFS para comunicação ou forte coordenação entre processos em diferentes computadores não é recomendado.

Segurança: a necessidade de segurança no NFS surgiu com a conexão de muitas intranets com a Internet. A integração do Kerberos com o NFS foi um passo importante nesse sentido. Outros desenvolvimentos recentes incluem a opção de usar uma implementação de RPC segura (RPC SEC_GSS, documentada no RFC 2203 [Eisler *et al.* 1997]) para autenticação, privacidade e segurança dos dados trans-

mitidos com operações de leitura e escrita. Existem muitas instalações que não implantam esses mecanismos e, portanto, são inseguras.

Eficiência: o desempenho medido de várias implementações de NFS, e sua adoção para uso em situações que geram cargas de trabalho muito pesadas, são indicações claras da eficiência com que o protocolo NFS pode ser implementado.

12.4 Estudo de caso: Andrew File System

Assim como o NFS, o AFS (Andrew File System) fornece acesso transparente a arquivos remotos compartilhados para programas UNIX executando em estações de trabalho. O acesso a arquivos AFS se dá por intermédio das primitivas de arquivo UNIX normais, permitindo que os programas UNIX existentes acessem arquivos AFS sem modificação, nem recompilação. O AFS é compatível com o NFS. Os servidores AFS contêm arquivos UNIX “locais”, mas o sistema de arquivos nos servidores é baseado no NFS; portanto, os arquivos são referenciados, em vez de por *i-nodes*, por manipuladores de arquivo do estilo NFS permitindo serem acessados de forma remota pelo NFS.

O AFS difere do NFS em seu projeto e implementação. As diferenças são atribuídas principalmente à identificação da escalabilidade como o objetivo de projeto mais importante. O AFS é feito para funcionar bem com números maiores de usuários ativos do que outros sistemas de arquivos distribuídos. A principal estratégia para se obter a escalabilidade é o uso de cache capazes de armazenar arquivos inteiros nos clientes. O AFS tem duas características de projeto incomuns:

Servir arquivos inteiros: o conteúdo inteiro de diretórios e de arquivos é transmitido para os computadores clientes pelos servidores AFS (no AFS-3, arquivos maiores do que 64 Kbytes são transferidos em porções de 64 Kbytes).

Cache de arquivos inteiros: quando uma cópia de um arquivo, ou uma porção, tiver sido transferida para um computador cliente, ela é armazenada em uma cache no disco local. A cache contém centenas dos arquivos usados mais recentemente nesse computador. Por ser em disco, a cache é permanente, sobrevivendo às reinicializações do computador cliente. Quando possível, são usadas cópias locais dos arquivos para atender às requisições *open* dos clientes, em detrimento das cópias remotas.

Cenário • Aqui está um cenário simples ilustrando o funcionamento do AFS:

1. Quando um processo de usuário em um computador cliente executa uma chamada de sistema *open* para um arquivo no espaço de arquivo compartilhado, e não existe uma cópia corrente do arquivo na cache local, o servidor que contém o arquivo é localizado e é enviada uma requisição de cópia do arquivo.
2. A cópia é armazenada no sistema de arquivos local do UNIX, no computador cliente; então, a cópia é aberta (com *open*) e o descritor de arquivo UNIX resultante é retornado para o cliente.
3. As operações *read*, *write* e outras, subsequentes no arquivo, executadas por processos no computador cliente, são realizadas na cópia local.
4. Quando o processo no cliente executa uma chamada de sistema *close*, se a cópia local tiver sido atualizada, seu conteúdo é enviado de volta para o servidor. O servidor atualiza o conteúdo do arquivo e os carimbos de tempo associados. A cópia que está no disco local do cliente é mantida, para o caso de ser novamente necessária, para um processo em nível de usuário na mesma estação de trabalho.

A seguir, vamos discutir o desempenho observado do AFS, mas podemos fazer aqui algumas observações e previsões gerais com base nas características de projeto descritas anteriormente:

- Para arquivos compartilhados que são raramente modificados (como os que contêm o código de comandos e bibliotecas do UNIX) e para arquivos que normalmente são acessados por apenas um usuário (como a maioria dos arquivos do diretório de base de um usuário e sua subárvore), é provável que as cópias armazenadas localmente na cache permaneçam válidas por longos períodos de tempo – no primeiro caso, porque eles não são atualizados, e no segundo, porque, se eles forem atualizados, a cópia atualizada estará na cache da estação de trabalho do proprietário. Essas classes de arquivo são responsáveis pela maioria dos acessos a arquivos.
- A cache local pode usar uma proporção substancial do espaço em disco de cada estação de trabalho, digamos, 100 megabytes. Normalmente, isso é suficiente para o estabelecimento de um conjunto de trabalho dos arquivos acessados por um único usuário. A provisão de armazenamento em cache suficiente para o estabelecimento de um conjunto de trabalho garante que os arquivos regularmente usados em uma determinada estação de trabalho sejam normalmente mantidos na cache até que sejam novamente necessários.
- A estratégia de projeto é baseada em algumas suposições sobre o tamanho de arquivo médio e máximo e sobre o caráter local da referência a arquivos nos sistemas UNIX. Essas suposições são derivadas de observações de cargas de trabalho típicas do UNIX em ambientes acadêmicos e outros [Satyanarayanan 1981; Ousterhout *et al.* 1985; Floyd 1986]. As observações mais importantes são:
 - Os arquivos são pequenos; a maioria tem menos de 10 Kbytes de tamanho.
 - As operações de leitura nos arquivos são muito mais comuns do que as de escrita (cerca de seis vezes mais comuns).
 - Acesso sequencial é comum, acesso aleatório é raro.
 - A maioria dos arquivos é lida e escrita por apenas um usuário. Quando um arquivo é compartilhado, normalmente é apenas um usuário que o modifica.
 - Os arquivos são referenciados em momentos específicos. Se um arquivo tiver sido referenciado recentemente, existe uma grande probabilidade de que ele seja referenciado novamente em um futuro próximo.

Essas observações foram usadas para delinear o projeto e a otimização do AFS e *não* para restringir a funcionalidade vista pelos usuários.

- O AFS funciona melhor com as classes de arquivo identificadas no primeiro ponto anterior. Há um tipo de arquivo importante que não se encaixa em nenhuma dessas classes – os bancos de dados são normalmente compartilhados por muitos usuários e, muitas vezes, são atualizados com bastante frequência. Os projetistas do AFS excluíram explicitamente de seus objetivos de projeto o suporte para bancos de dados, dizendo que as restrições impostas por diferentes estruturas de atribuição de nomes (isto é, acesso baseado no conteúdo), a necessidade de granularidade fina para acesso aos dados, o controle de concorrência e a atomicidade das atualizações dificultam o projeto de um sistema de banco de dados distribuído que também seja um sistema de arquivos distribuído. Eles argumentam que os requisitos necessários para bancos de dados distribuídos devem ser tratados separadamente [Satyanarayanan 1989a].

12.4.1 Implementação

O cenário anterior ilustra o funcionamento do AFS, mas deixa sem resposta muitas perguntas sobre sua implementação. Dentre as mais importante estão:

- Como o AFS obtém o controle quando uma chamada de sistema *open* (ou *close*) se referindo a um arquivo no espaço de arquivos compartilhado é feita por um cliente?
- Como é localizado o servidor que contém o arquivo solicitado?
- Que espaço em cache é alocado para arquivos nas estações de trabalho?
- Como o AFS garante que as cópias dos arquivos colocadas na cache estão atualizadas quando os arquivos podem ser modificados por vários clientes?

Responderemos a essas perguntas a seguir.

O AFS é implementado com base em dois componentes de *software* que existem como processos UNIX, em nível de usuário, chamados *Vice* e *Venus*. A Figura 12.11 mostra a distribuição dos processos *Vice* e *Venus*. *Vice* é o nome dado ao *software* servidor executado em cada computador servidor, e *Venus* é um processo executado em cada computador cliente e corresponde ao módulo cliente em nosso modelo abstrato.

Os arquivos disponíveis para processos de usuário em execução nas estações de trabalho são *locais* ou *compartilhados*. Os arquivos locais são tratados como arquivos normais do UNIX. Eles são armazenados em um disco da estação de trabalho e estão disponíveis somente para processos de usuário locais. Os arquivos compartilhados são armazenados em servidores, e cópias deles são armazenadas na cache nos discos locais das estações de trabalho. O espaço de nomes visto pelos processos de usuário está ilustrado na Figura 12.12. Trata-se de uma hierarquia de diretórios UNIX convencional, com uma subárvore específica (chamada *cmu*), contendo todos os arquivos compartilhados. Essa divisão do espaço de nomes de arquivo em arquivos locais e compartilhados leva a certa perda de transparência de localização, mas isso dificilmente é perceptível para usuários que não sejam os administradores de sistema. Os arquivos locais são usados apenas por arquivos temporários (*/tmp*) e por processos que são fundamentais para a inicialização da estação de trabalho. Outros arquivos padrão do UNIX (como aqueles normalmente encontrados em */bin*, */lib*, etc.) são

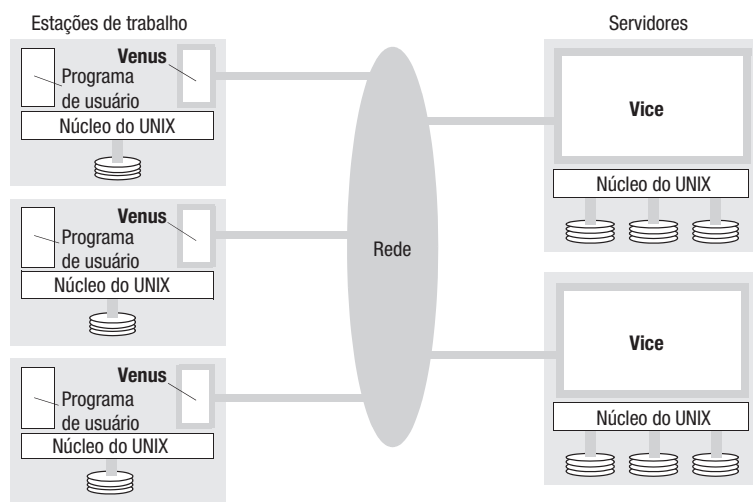


Figura 12.11 Distribuição de processos no Andrew File System.

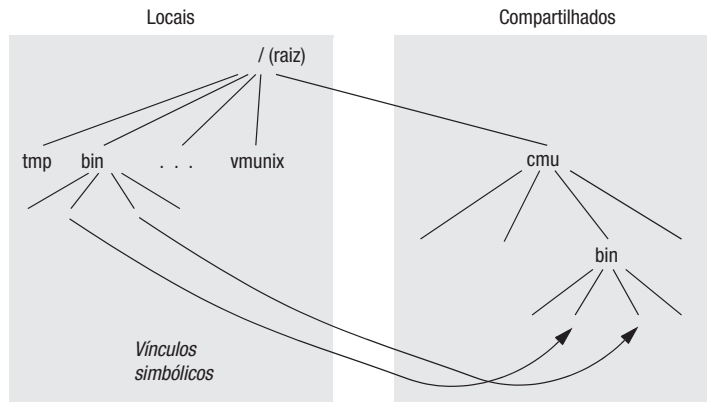


Figura 12.12 Espaço de nomes de arquivo visto pelos clientes do AFS.

implementados como vínculos simbólicos de diretórios locais para arquivos mantidos no espaço compartilhado. Os diretórios dos usuários ficam no espaço compartilhado, permitindo que os usuários acessem seus arquivos a partir de qualquer estação de trabalho.

O núcleo UNIX, em cada estação de trabalho e servidor, é uma versão modificada do UNIX BSD. As modificações são feitas para interceptar chamadas de sistema de arquivos *open*, *close* e algumas outras, quando elas se referem a arquivos do espaço de nomes compartilhado e as passam para o processo Venus no computador cliente (ilustrado na Figura 12.13). Outra modificação do núcleo é incluída por motivos de desempenho e será descrita posteriormente.

Uma partição do disco local de cada estação de trabalho é usada como a cache e conterá as cópias dos arquivos do espaço compartilhado. Venus gerencia a cache. Se, ao obter um novo arquivo a partir do servidor, a partição de cache estiver cheia, Venus remove os arquivos usados menos recentemente para abrir espaço. Normalmente, a cache da estação de trabalho é grande o suficiente para acomodar várias centenas de arquivos de tamanho médio. Isso torna a estação de trabalho amplamente independente dos servidores Vice, uma vez que um conjunto dos arquivos de trabalho do usuário corrente e dos arquivos de sistema usados frequentemente foram colocados na cache.

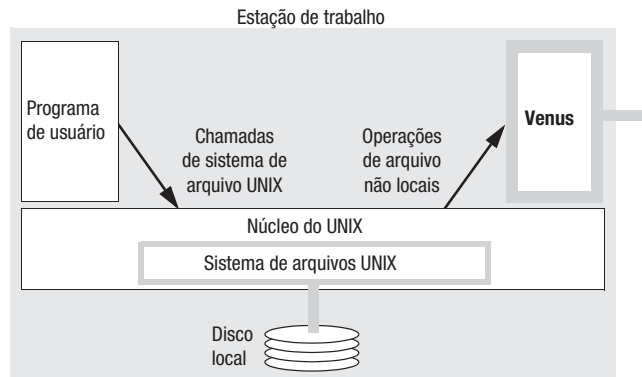


Figura 12.13 Interceptação de chamada de sistema no AFS.

O AFS é semelhante ao modelo de serviço de arquivos abstrato, descrito na Seção 12.2, sob os seguintes aspectos:

- Um serviço de arquivos simples é implementado pelos servidores Vice, e a estrutura de diretório hierárquica exigida pelos programas de usuário do UNIX é implementada pelo conjunto de processos Venus nas estações de trabalho.
- Cada arquivo e diretório no espaço de arquivo compartilhado é identificado por um identificador de arquivo (*fid* – *file identifier*) único de 96 bits, semelhante a um UFID. Os processos Venus traduzem os nomes de caminho, fornecidos pelos clientes, em *fids*.

Os arquivos são agrupados em *volumes* para facilitar a localização e a movimentação. Geralmente, os volumes são menores do que os sistemas de arquivos UNIX, que são a unidade de agrupamento de arquivos do NFS. Por exemplo, os arquivos pessoais de cada usuário geralmente estão localizados em um volume separado. Outros volumes são alocados para binários do sistema, documentação e códigos de bibliotecas.

A representação dos *fids* inclui o número do volume que contém o arquivo (compare com o *identificador de grupo de arquivos* nos UFIDs), um manipulador de arquivo NFS identificando o arquivo dentro do volume (conforme o *número de arquivo* nos UFIDs) e um *elemento de exclusividade* (*uniquifier*) para garantir que os identificadores de arquivo não sejam reutilizados:

32 bits	32 bits	32 bits
Número do volume	Manipulador de arquivo	Elemento de exclusividade

Os programas de usuário utilizam nomes de caminho convencionais do UNIX para referenciar arquivos, mas o AFS usa *fids* na comunicação entre os processos Venus e Vice. Os servidores Vice aceitam requisições apenas em termos de *fids*. O Venus traduz os nomes de caminho fornecidos pelos clientes em *fids*, usando uma pesquisa passo a passo para obter as informações dos diretórios de arquivo mantidos nos servidores Vice.

A Figura 12.14 descreve as ações executadas pelo Vice, pelo Venus e pelo núcleo do UNIX, quando um processo de usuário executa cada uma das chamadas de sistema mencionadas em nosso cenário delineado anteriormente. A *promessa de callback*, mencionada na figura, é um mecanismo para garantir que as cópias dos arquivos armazenadas em cache sejam atualizadas quando outro cliente fechar o mesmo arquivo após modificá-lo. Esse mecanismo será discutido na próxima seção.

12.4.2 Consistência da cache

Quando o Vice fornece uma cópia de um arquivo para um processo Venus, ele também fornece uma *promessa de callback* – um tipo de tíquete (*token*) emitido pelo servidor Vice que é o depósito do arquivo, garantindo que notificará o processo Venus quando qualquer outro cliente modificar o arquivo. As promessas de *callback* são armazenadas junto aos arquivos na cache em disco da estação de trabalho e têm dois estados: *válida* ou *cancelada*. Quando um servidor realiza uma requisição de atualização de um arquivo, ele notifica todos os processos Venus para os quais tem promessas de *callback* emitidas, enviando um *callback* para cada um – um *callback* é uma chamada de procedimento remoto feita por um servidor para um processo Venus. Quando o processo Venus recebe um *callback*, ele configura o *token* de *promessa de callback* do arquivo em questão como *cancelado*.

Quando Venus trata uma operação *open* em nome de um cliente, ele verifica a cache. Se o arquivo solicitado for encontrado na cache, seu *token* é verificado. Se o valor for *cancelado*, uma cópia nova do arquivo deve ser buscada no servidor Vice, mas se o

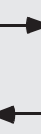
Processo de usuário	Núcleo do UNIX	Venus	Rede	Vice
<i>open</i> (FileName, mode)	Se <i>FileName</i> se refere a um arquivo no espaço de arquivos compartilhado, passa a requisição para Venus. Abre o arquivo local e retorna o descritor de arquivo para o aplicativo.	Verifica a lista de arquivos na cache local. Se não estiver presente ou não houver uma promessa de <i>callback</i> válida, envia uma requisição do arquivo para o servidor Vice que é o depositório do volume que contém o arquivo. Coloca a cópia do arquivo no sistema de arquivos local, insere seu nome local na lista da cache e retorna o nome local para o UNIX.		Transfere uma cópia do arquivo e uma <i>promessa de callback</i> para a estação de trabalho. Registra a promessa de <i>callback</i> .
<i>read</i> (FileDescriptor, Buffer, length)	Executa uma operação de leitura UNIX normal na cópia local.			
<i>write</i> (FileDescriptor, Buffer, length)	Executa uma operação de escrita UNIX normal na cópia local.			
<i>close</i> (FileDescriptor)	Fecha a cópia local e notifica Venus de que o arquivo foi fechado.	Se a cópia local tiver sido alterada, envia uma cópia para o servidor Vice que é o depositório do arquivo.		Substitui o conteúdo do arquivo e envia um <i>callback</i> para todos os outros clientes que contêm <i>promessas de callback</i> no arquivo.

Figura 12.14 Implementação de chamadas de sistema de arquivos no AFS.

token for *válido*, então a cópia armazenada na cache poderá ser aberta e usada, sem referência ao Vice.

Quando uma estação de trabalho é reiniciada após uma falha ou desligamento, Venus tenta manter o máximo possível dos arquivos armazenados na cache no disco local, mas não pode presumir que os *tokens* de promessa de *callback* estejam corretos, pois alguns *callbacks* podem ter sido perdidos. Portanto, antes do primeiro uso de cada arquivo, ou diretório armazenados na cache após uma reinicialização, Venus gera uma requisição de validação de cache contendo o carimbo de tempo da modificação do arquivo para o servidor que é o depositório do arquivo. Se o carimbo de tempo estiver atualizado, o servidor responderá com *válido* e o *token* será reinstalado. Se o carimbo de tempo mostrar que o arquivo está desatualizado, então o servidor responderá com *cancelado* e o *token* será configurado como *cancelado*. Os *callbacks* devem ser renovados antes de uma operação *open*, caso um tempo *T* (normalmente, da ordem de alguns minutos) tenha decorrido desde que o arquivo foi posto na cache sem ter havido comunicação do servidor. Isso serve para tratar de possíveis falhas de comunicação, as quais podem resultar na perda de mensagens de *callback*.

Esse mecanismo baseado em *callbacks* para manutenção da consistência da cache foi adotado como a estratégia que oferecia maior escalabilidade, de acordo com a avaliação no protótipo (AFS-1) de um mecanismo baseado em carimbos de tempo semelhante àquele usado no NFS. No AFS-1, um processo Venus, contendo uma cópia de um arquivo na cache,

<i>Fetch(fid) → attr, data</i>	Retorna os atributos (status) e, opcionalmente, o conteúdo do arquivo identificado pelo <i>fid</i> e registra nele uma promessa de <i>callback</i> .
<i>Store(fid, attr, data)</i>	Atualiza os atributos e (opcionalmente) o conteúdo de um arquivo especificado.
<i>create() → fid</i>	Cria um novo arquivo e registra nele uma promessa de <i>callback</i> .
<i>Remove(fid)</i>	Exclui o arquivo especificado.
<i>SetLock(fid, mode)</i>	Estabelece uma trava (<i>lock</i>) no arquivo ou diretório especificado. O modo da trava pode ser compartilhado ou exclusivo. As travas que não são removidas expiram após 30 minutos.
<i>ReleaseLock(fid)</i>	Destrava o arquivo ou diretório especificado.
<i>RemoveCallback(fid)</i>	Informa o servidor que um processo Venus removeu um arquivo de sua cache.
<i>BreakCallback(fid)</i>	Esta chamada é feita por um servidor Vice para um processo Venus. Ela cancela a promessa de <i>callback</i> no arquivo identificado por <i>fid</i> .

Nota: As operações de diretório e administrativas (Rename, Link, Makedir, Removedir, GetTime, CheckToken, etc.) não são mostradas.

Figura 12.15 Os principais componentes da interface do serviço Vice.

interroga o processo Vice em cada operação *open* para determinar se o carimbo de tempo na cópia local está de acordo com a do servidor. A estratégia baseada em *callback* tem maior escalabilidade porque resulta na comunicação entre cliente e servidor e na atividade do servidor somente quando o arquivo tiver sido atualizado, enquanto a estratégia de carimbo de tempo causa uma interação cliente-servidor em cada operação *open*, mesmo quando existe uma cópia local válida. Como a maioria dos arquivos não é acessada concorrentemente, e as operações *read* predominam sobre as operações *write* na maioria dos aplicativos, o mecanismo de *callback* causa uma importante redução no número de interações cliente-servidor.

O mecanismo de *callback* usado no AFS-2, e em versões posteriores do AFS, exige que os servidores Vice mantenham algum estado em nome de seus clientes Venus, ao contrário do que ocorre no AFS-1, no NFS e em nosso modelo de serviço de arquivos. O estado depende do cliente e consiste em uma lista dos processos Venus para os quais foram emitidas promessas de *callback* para cada arquivo. Essas listas de *callback* devem ser mantidas no caso de falhas do servidor – elas são mantidas nos discos do servidor e atualizadas com operações atômicas.

A Figura 12.15 mostra as chamadas de RPC fornecidas pelos servidores AFS para operações em arquivos (isto é, a interface fornecida pelos servidores AFS para processos Venus).

Semântica de atualização • O objetivo desse mecanismo de consistência de cache é obter a melhor aproximação possível da semântica de arquivo de cópia única, sem uma degradação séria de desempenho. Uma implementação estrita da semântica de cópia única, para primitivas de acesso a arquivo do UNIX, exigiria que os resultados de cada operação *write* em um arquivo fossem distribuídos para todos os *sites* que o contêm em suas caches, antes que mais acessos pudessem ocorrer. Isso não pode ser feito em sistemas de larga escala; em vez disso, o mecanismo de promessa de *callback* mantém uma aproximação da semântica de cópia única.

Para o AFS-1, a semântica de atualização pode ser declarada formalmente em termos muito simples. Para um cliente C operando em um arquivo F , cujo depositório é um servidor S , as seguintes garantias de atualização das cópias de F são mantidas:

após uma operação *open* bem-sucedida: $latest(F, S)$

após uma operação *open* malsucedida: $failure(S)$

após uma operação *close* bem-sucedida: $updated(F, S)$

após uma operação *close* malsucedida: $failure(S)$

onde $latest(F, S)$ denota uma garantia de que o valor corrente de F , em C , é igual ao valor em S , $failure(S)$ denota que a operação *open* (ou *close*) não foi efetuada em S (e a falha pode ser detectada por C) e $updated(F, S)$ denota que o valor de C de F foi propagado com êxito para S .

Para o AFS-2, a garantia de atualização de *open* é ligeiramente mais fraca, e a declaração formal da garantia correspondente é mais complexa. Isso porque um cliente pode abrir uma cópia antiga de um arquivo após ele ter sido atualizado por outro cliente. Isso ocorre se uma mensagem de *callback* é perdida, por exemplo, como resultado de uma falha da rede. No entanto, existe um tempo máximo T durante o qual um cliente pode permanecer sem saber da existência de uma versão mais recente de um arquivo. Portanto, temos a seguinte garantia:

após uma operação *open* bem-sucedida: $latest(F, S, 0)$
ou $(lostCallback(S, T) \text{ e } inCache(F) \text{ e } latest(F, S, T))$

onde $latest(F, S, T)$ denota que a cópia de F vista pelo cliente não está há mais do que T segundos desatualizada, $lostCallback(S, T)$ denota que uma mensagem de *callback* de S para C foi perdida em algum momento durante os últimos T segundos e $inCache(F)$, que o arquivo F estava na cache em C antes que a operação *open* fosse tentada. A declaração formal anterior expressa o fato de que a cópia colocada na cache de F em C , após uma operação *open*, é a versão mais recente presente no sistema, ou que uma mensagem de *callback* foi perdida (devido a uma falha de comunicação) e a versão que já estava na cache foi utilizada. A versão colocada na cache não estará há mais do que T segundos desatualizada (T é uma constante do sistema representando o intervalo em que as promessas de *callback* devem ser renovadas. Na maioria das instalações, o valor de T é configurado em cerca de 10 minutos.)

De acordo com seu objetivo – fornecer um serviço de arquivo distribuído de larga escala, compatível com o UNIX –, o AFS não fornece mais nenhum mecanismo para o controle de atualizações concorrentes. O algoritmo de consistência de cache descrito anteriormente entra em ação somente nas operações *open* e *close*. Uma vez que um arquivo tenha sido aberto, o cliente pode acessar e atualizar a cópia local da maneira que quiser, sem o conhecimento de quaisquer processos em outras estações de trabalho. Quando o arquivo é fechado, uma cópia é retornada para o servidor substituindo a versão corrente.

Se clientes, em diferentes estações de trabalho, abrem, escrevem e fecham (com as operações *open*, *write* e *close*) o mesmo arquivo concorrentemente, todas as atualizações, menos a resultante da última operação *close*, serão perdidas silenciosamente (nenhum relatório de erro é fornecido). Os clientes devem implementar o controle de concorrência independentemente, caso exijam isso. Por outro lado, quando dois processos clientes na mesma estação de trabalho abrem um arquivo, eles compartilham a mesma cópia colocada na cache, e as atualizações são realizadas da maneira normal do UNIX – bloco por bloco.

Embora a semântica de atualização seja diferente, dependendo das localizações dos processos concorrentes que estão acessando um arquivo, e não seja precisamente igual àquela fornecida pelo sistema de arquivos padrão do UNIX, elas são suficientemente parecidas para que a maioria dos programas UNIX existentes funcione corretamente.

12.4.3 Outros aspectos

Modificações no núcleo do UNIX • Notamos que o servidor Vice é um processo usuário executando no computador servidor e o servidor é dedicado ao estabelecimento de um serviço AFS. O núcleo do UNIX nos computadores que usam AFS é alterado para que o Vice possa efetuar operações de arquivo em termos de manipuladores de arquivo, em vez dos descritores de arquivo convencionais do UNIX. Essa é a única modificação no núcleo exigida pelo AFS e é necessária se o Vice não precisar manter o estado do cliente (como os descritores de arquivo).

Banco de dados de localização • Cada servidor contém uma cópia de um banco de dados de localização totalmente replicado, fornecendo um mapeamento de nomes de volume para servidores. Quando um volume é movido, podem ocorrer imprecisões temporárias nesse banco de dados, mas elas são inofensivas, pois o encaminhamento de informações é deixado para trás, no servidor a partir do qual o volume é movido.

Threads • As implementações de Vice e Venus utilizam uma biblioteca de *threads* não preemptiva para permitir que as requisições sejam processadas concorrentemente no cliente (em que vários processos de usuário podem ter requisições de acesso a arquivo em andamento concorrentemente) e no servidor. No cliente, as tabelas que descrevem o conteúdo da cache e o banco de dados de volumes são mantidos na memória compartilhada entre as *threads* do Venus.

Réplicas somente de leitura • Os volumes que contêm arquivos lidos frequentemente, mas raramente modificados, como os diretórios de comandos de sistema UNIX */bin* e */usr/bin* e o diretório de páginas de manual */man*, podem ser replicados como volumes somente de leitura em vários servidores. Quando isso é feito, existe apenas uma réplica para leitura e escrita, e todas as atualizações são direcionadas a ela. A propagação das alterações para as réplicas somente de leitura é realizada após a atualização por parte de um procedimento operacional explícito. As entradas no banco de dados de localização para volumes replicados dessa maneira são do tipo uma para muitas, e o servidor para cada requisição de cliente é selecionado de acordo com as cargas e a acessibilidade do servidor.

Transferências de grandes volumes • O AFS transfere arquivos entre clientes e servidores em porções de 64 Kbytes. O uso de um pacote desse tamanho é uma ajuda importante para o desempenho, minimizando o efeito da latência da rede. Assim, o projeto do AFS permite que o uso da rede seja otimizado.

Uso de cache parcial de arquivo • A necessidade de transferir o conteúdo inteiro dos arquivos para os clientes, mesmo quando o requisito do aplicativo é ler apenas uma pequena parte do arquivo, é uma fonte de ineficiência óbvia. A versão 3 do AFS eliminou esse requisito, permitindo que os dados dos arquivos sejam transferidos e colocados na cache em blocos de 64 Kbytes, enquanto ainda mantém a semântica de consistência e outras características do protocolo AFS.

Desempenho • O principal objetivo do AFS é a escalabilidade; portanto, seu desempenho com grandes números de usuários é de particular interesse. Howard *et al.* [1988] fornecem detalhes de extensivas medidas de desempenho, que foram feitas usando um *benchmark* especialmente desenvolvido para o AFS, que posteriormente foi usado para a avaliação de sistemas de arquivos distribuídos. Evidentemente, o uso de cache para o arquivo inteiro e o protocolo de *callback* levaram a cargas substancialmente menores nos servidores. Satyanarayanan [1989a] informa que foi medida uma carga de servidor de 40%, com 18 clientes executando um *benchmark* padrão, contra uma carga de 100% para o NFS executando o mesmo *benchmark*. Satyanarayanan atribui grande parte da vantagem do desempenho do AFS à redução na carga do servidor derivada do uso de *callbacks* para notificar os clientes

sobre atualizações nos arquivos, se comparada ao mecanismo de carimbos de tempo usado no NFS para verificar a validade das páginas colocadas na cache nos clientes.

Suporte remoto • A versão 3 do AFS suporta várias células administrativas, cada uma com seus próprios servidores, clientes, administradores de sistema e usuários. Cada célula é um ambiente completamente autônomo, mas uma federação de células pode cooperar para apresentar aos usuários um espaço de nomes de arquivo uniforme e transparente. O sistema resultante foi amplamente implantado pela Transarc Corporation, e foi publicado um levantamento detalhado dos padrões de utilização e desempenho resultantes [Spasojevic e Satyanarayanan 1996]. O sistema foi instalado em mais de 1.000 servidores, em mais de 150 *sites*. O levantamento mostrou taxas de utilização da cache no intervalo de 96–98% para acessos a uma amostra de 32.000 volumes de arquivo, contendo 200 Gbytes de dados.

12.5 Aprimoramentos e mais desenvolvimentos

Vários avanços foram feitos no projeto de sistemas de arquivos distribuídos desde o surgimento do NFS e do AFS. Nesta seção, descreveremos os avanços que melhoram o desempenho, a disponibilidade e a escalabilidade dos sistemas de arquivos distribuídos convencionais. Avanços mais radicais são descritos em outras partes deste livro, incluindo a manutenção da consistência em sistemas de arquivos de leitura e escrita replicados para suportar operação desconectada e alta disponibilidade, nos sistemas Bayou e Coda (Seções 18.4.2 e 18.4.3), e uma arquitetura com grande escalabilidade para a implantação de fluxos de dados em tempo real com garantia de qualidade, no servidor de arquivos de vídeo Tiger (Seção 20.6.1).

Melhorias no NFS • Vários projetos de pesquisa trataram da questão da semântica de atualização de cópia única, ampliando o protocolo NFS para incluir as operações *open* e *close* e adicionar um mecanismo de *callback* para permitir que o servidor notifique os clientes sobre a necessidade de invalidar entradas da cache. Descreveremos aqui dois desses trabalhos; seus resultados parecem indicar que esses aprimoramentos podem ser acomodados sem complexidade ou custos de comunicação extras.

Alguns trabalhos recentes da Sun, e de outros desenvolvedores de NFS, foram direcionados de forma a tornar os servidores NFS mais acessíveis e úteis em redes de longa distância. Embora o protocolo HTTP, suportado pelos servidores Web, ofereça um método eficiente e com alta escalabilidade para tornar arquivos inteiros disponíveis para os clientes por meio da Internet, ele é menos útil para programas aplicativos que exigem acesso a partes de arquivos grandes, ou para aqueles que atualizam partes de arquivos. O desenvolvimento do WebNFS (descrito a seguir) possibilita que programas aplicativos se tornem clientes de servidores NFS, em qualquer parte da Internet (usando o protocolo NFS diretamente, em vez de usar indiretamente, por meio de um módulo do núcleo). Isso, junto a bibliotecas apropriadas da linguagem Java e de outras linguagens de programação de rede, deve oferecer a possibilidade de implementar aplicativos de Internet que compartilhem dados diretamente, como jogos para vários usuários ou clientes de grandes bancos de dados dinâmicos.

Obtenção da semântica de atualização de cópia única: a arquitetura de servidor sem estado do NFS trouxe grandes vantagens na robustez e na facilidade de implementação do NFS, mas impossibilita a obtenção de uma semântica de atualização de cópia única precisa (não há garantia de que os efeitos de gravações concorrentes feitas por diferentes clientes no mesmo arquivo sejam os mesmos de um sistema UNIX único, quando vários processos gravam em um arquivo local). Ela também impede o uso de *callbacks* notificando os clientes sobre alterações em arquivos, e isso resulta em frequentes requisições de *getattr* dos clientes para verificar a existência de modificação no arquivo.

Dois sistemas de pesquisa foram desenvolvidos para tratar desses inconvenientes. O Spritely NFS [Srinivasan e Mogul 1989, Mogul 1994] é uma evolução do sistema de arquivos desenvolvido na Berkeley para o sistema operacional distribuído Sprite [Nelson *et al.* 1988]. O Spritely NFS é uma implementação do protocolo NFS com a inclusão de chamadas *open* e *close*. Os módulos dos clientes devem enviar uma operação *open* quando um processo local usuário abre um arquivo que está no servidor. Os parâmetros da operação *open* do Sprite especificam um modo (leitura, gravação ou ambos) e incluem contagens do número de processos locais que têm o arquivo aberto correntemente para leitura e para escrita. Analogamente, quando um processo local fecha um arquivo remoto, uma operação *close* é enviada para o servidor, com contagens atualizadas de leitores e escritores. O servidor registra esses números em uma *tabela de arquivos abertos*, com o endereço IP e o número de porta do cliente.

Quando o servidor recebe uma operação *open*, ele verifica a *tabela de arquivos abertos* de outros clientes que têm o mesmo arquivo aberto e envia mensagens de *callback* para esses clientes, instruindo-os a modificar suas estratégias de uso de cache. Se a operação *open* especificar modo de escrita, então ela falhará, caso qualquer outro cliente tenha o arquivo aberto para escrita. Outros clientes que tiverem o arquivo aberto para leitura serão instruídos a invalidar as partes do arquivo colocadas na cache de forma local.

Para operações *open* que especificam o modo de leitura, o servidor envia uma mensagem de *callback* para todo cliente que esteja escrevendo, dizendo a ele para parar de usar a cache (isto é, para usar um modo de operação rigorosamente de escrita direta – *write-through*), e instrui a todos os clientes que estão lendo para que parem de colocar o arquivo na cache (para que todas as chamadas de leitura locais resultem em uma requisição para o servidor).

Essas medidas resultam em um serviço de arquivo que mantém a semântica de atualização de cópia única do UNIX às custas do transporte de algum estado relacionado ao cliente no servidor. Elas também permitem alguns ganhos de eficiência no tratamento de escritas na cache. Se o estado relacionado ao cliente for mantido em memória volátil no servidor, ele será vulnerável às falhas do servidor. O Spritely NFS implementa um protocolo de recuperação que interroga uma lista de clientes que abriram arquivos recentemente no servidor, para recuperar a *tabela de arquivos abertos* inteira. A lista de clientes é armazenada no disco, sendo atualizada de forma relativamente rara, e é pessimista – ela pode incluir, com segurança, mais clientes do que aqueles que tinham arquivos abertos no momento da falha. Clientes defeituosos também podem resultar em excesso de entradas na *tabela de arquivos abertos*, mas elas serão removidas quando o cliente reiniciar.

Quando o Spritely NFS foi avaliado com relação ao NFS, versão 2, apresentou uma melhoria de desempenho modesta. Isso se deu graças ao uso melhorado da escrita em cache. Mudanças no NFS, versão 3, provavelmente resultariam pelo menos em uma melhoria igual, mas os resultados do projeto Spritely NFS certamente indicam que é possível obter semântica de atualização de cópia única sem perda de desempenho substancial, embora às custas de alguma complexidade de implementação extra nos módulos cliente e servidor e da necessidade de um mecanismo de recuperação para restaurar o estado após uma falha de servidor.

NQNFS: o projeto NQNFS (Not Quite NFS) [Macklem 1994] tinha objetivos semelhantes ao Spritely NFS – adicionar consistência de cache mais precisa ao protocolo NFS e melhorar o desempenho por intermédio do uso melhor da cache. Um servidor NQNFS mantém estado similar relacionado ao cliente, a respeito de arquivos abertos, mas utiliza arrendamentos (Seção 5.4.3) para ajudar na recuperação, após uma falha de servidor. O servidor estabelece um limite superior para o tempo durante o qual um cliente pode manter um arrendamento para um arquivo aberto. Se o cliente quiser continuar além desse tempo, precisará renovar o arrendamento. *Callbacks* são usadas de maneira semelhante ao Spritely

NFS, para pedir aos clientes que invalidem suas caches quando ocorrer uma requisição de escrita, mas se os clientes não responderem, o servidor simplesmente esperará até que seus arrendamentos expirem, antes de responder à nova requisição de escrita.

WebNFS: o advento da Web e dos *applets* Java levou ao reconhecimento, por parte da equipe de desenvolvimento do NFS e outros, de que alguns aplicativos de Internet poderiam tirar proveito do acesso direto aos servidores NFS sem grande parte das sobrecargas associadas à simulação das operações de arquivo do UNIX incluídas nos clientes NFS padrão.

O objetivo do WebNFS (descrito nas RFCs 2055 e 2056 [Callaghan 1996a, 1996b]) é permitir que navegadores Web, programas Java e outros aplicativos interajam diretamente com um servidor NFS para acessar arquivos que são publicados, usando um *manipulador de arquivo público* para acessar arquivos relativos a um diretório raiz público. Esse modo de uso anula o serviço *mount* e o serviço mapeador de porta (descritos no Capítulo 5). Os clientes WebNFS interagem com um servidor NFS em um número de porta bem conhecido (2049). Para acessar arquivos pelo nome de caminho, eles emitem requisições de *lookup* usando um manipulador de arquivo público. O manipulador de arquivo público tem um valor interpretado de forma especial pelo sistema de arquivos virtual no servidor. Devido à alta latência das redes de longa distância, é usada uma variante da operação *lookup* para pesquisar um nome de caminho de várias partes em uma única requisição.

Assim, o WebNFS permite o desenvolvimento de clientes que acessam partes de arquivos armazenados em servidores NFS de *sites* remotos com sobrecargas de configuração mínimas. Há suporte para controle de acesso e autenticação, mas, em muitos casos, o cliente solicitará apenas acesso de leitura para arquivos públicos e, nesse caso, a opção de autenticação poderá ser desativada. Ler uma parte de um único arquivo localizado em um servidor NFS que suporta WebNFS exige o estabelecimento de uma conexão TCP e duas chamadas de RPC – uma para a operação *lookup* de várias partes e outra para a operação *read*. O tamanho do bloco de dados lido não é limitado pelo protocolo NFS.

Por exemplo, um serviço de previsão de tempo poderia publicar um arquivo em seu servidor NFS, contendo um grande banco de dados de dados climáticos atualizados frequentemente, com um URL do tipo:

nfs://data.weather.gov/weatherdata/global.data

Um cliente *WeatherMap* interativo, que mostrasse mapas do clima, poderia ser construído em Java, ou em qualquer outra linguagem, que ofereça suporte a biblioteca de procedimentos WebNFS. O cliente lê apenas as partes do arquivo */weatherdata/global.data* que são necessárias para construir os mapas específicos solicitados por um usuário, enquanto um aplicativo semelhante, que usasse HTTP para acessar um servidor de dados climáticos, teria que transferir o banco de dados inteiro para o cliente, ou solicitar o suporte de um programa servidor de propósito específico para fornecer os dados solicitados.

NFS versão 4: uma nova versão do protocolo NFS foi apresentada em 2000. Os objetivos do NFS versão 4 estão descritos no RFC 2624 [Shepler 1999] e no livro de Brent Callaghan [Callaghan 1999]. Assim como o WebNFS, ela pretende facilitar o uso do NFS em redes de longa distância e em aplicativos de Internet. Ela inclui os recursos do WebNFS, mas a introdução de um novo protocolo também apresenta a oportunidade de fazer aprimoramentos mais radicais. (O WebNFS ficou restrito às alterações no servidor que não envolviam essa adição de novas operações no protocolo.)

O NFS versão 4 explora os resultados que surgiram das pesquisas feitas no projeto do servidor de arquivos na última década, como o uso de *callbacks* ou de arrendamentos para manter a consistência. O NFS versão 4 suporta recuperação dinâmica de falhas do servidor, permitindo que os sistemas de arquivos sejam movidos para novos servidores

de forma transparente. A capacidade de mudança de escala é aprimorada pelo uso de servidores *proxies* de uma maneira análoga ao seu uso na Web.

Aprimoramentos do AFS • Mencionamos que o DCE/DFS, o sistema de arquivos distribuído incluído no *Distributed Computing Environment*, da Open Software Foundation [www.opengroup.org], era baseado no Andrew File System. O projeto do DCE/DFS vai além do AFS, particularmente em sua estratégia para consistência da cache. No AFS, os *callbacks* são gerados apenas quando o servidor recebe uma operação *close* para um arquivo que foi atualizado. O DFS adotou uma estratégia semelhante para o Sprite NFS e para o NQNFS, isto é, gera *callbacks* assim que um arquivo é atualizado. Para atualizar um arquivo, um cliente deve obter um *token write* do servidor, especificando um intervalo de bytes que o cliente pode atualizar no arquivo. Quando um *token write* é solicitado, os clientes que contêm cópias do mesmo arquivo para leitura recebem *callbacks* de revogação. *Tokens* de outros tipos são usados para obter consistência de atributos de arquivo colocados na cache e outros metadados. Todos os *tokens* têm um tempo de vida associado, e os clientes devem renová-los após sua expiração.

Aprimoramentos na organização do armazenamento • Houve um progresso considerável na organização dos dados de arquivos armazenados em discos. O impulso para grande parte desse trabalho surgiu das cargas maiores e da maior confiabilidade que os sistemas de arquivos distribuídos precisam suportar, e ele resultou em sistemas de arquivos com desempenho substancialmente melhor. Os principais resultados desse trabalho são:

RAID (Redundant Arrays of Inexpensive Disks): trata-se de um modo de armazenamento [Patterson *et al.* 1988, Chen *et al.* 1994] no qual blocos de dados são segmentados em porções de tamanho fixo e armazenados em “tiras” em vários discos (*striping*), junto a códigos redundantes de correção de erro que permitem aos blocos de dados serem completamente reconstruídos e, no caso de falhas de disco, as operações de leitura e escrita continuarem normalmente. O RAID também produz um desempenho consideravelmente melhor do que um único disco, pois as tiras (*strips*) que constituem um bloco são lidas e escritas concorrentemente.

LFS (Log-structured file storage): assim como o Sprite NFS, esta técnica foi originada no projeto do sistema operacional distribuído Sprite, em Berkeley [Rosenblum e Ousterhout 1992]. Os autores observaram que, quando grandes quantidades de memória principal eram empregadas como cache em servidores de arquivos, melhor era o desempenho da leitura, porém o das operações de escrita continuava medíocre. Isso era proveniente das altas latências associadas à escrita de blocos de dados individuais no disco e às atualizações associadas nos blocos de metadados (isto é, os blocos conhecidos como *i-nodes*, que contêm atributos de arquivo e ponteiros para os blocos em um arquivo).

A solução do LFS é acumular um conjunto de escritas na memória e, então, efetivá-las no disco em grandes segmentos, adjacentes e de tamanho fixo. Eles são chamados de *segmentos de log* porque os blocos de dados e metadados são armazenados rigorosamente na ordem em que foram atualizados. Um segmento de *log* tem 1 Mbyte, ou mais, de tamanho, e é armazenado em uma única trilha do disco, eliminando as latências do cabeçote do disco associadas à escrita de blocos individuais. Os blocos de dados e metadados, novos ou atualizados, são sempre escritos no disco exigindo a manutenção de um mapa dinâmico (na memória, com um *backup* persistente) apontando para os blocos de *i-nodes*. Também é exigida a coleta de lixo de blocos antigos, com compactação de blocos “ativos”, para deixar áreas de armazenamento adjacentes livres para o armazenamento de segmentos de *log*. Este último é um processo bastante complexo; ele é executado em *background* por um componente cha-

mado *limpador* (*cleaner*). Alguns algoritmos limpadores sofisticados foram desenvolvidos para isso, com base nos resultados de simulações.

Apesar desses custos extras, o ganho de desempenho geral é significativo; Rosenblum e Ousterhout mediram um desempenho de escrita de até 70% da largura de banda disponível no disco, comparado com menos de 10% para um sistema de arquivos UNIX convencional. A estrutura do *log* também simplifica a recuperação após falhas do servidor. O sistema de arquivos Zebra [Hartman e Ousterhout 1995], desenvolvidos como uma continuação do trabalho original do LFS, combina escritas estruturadas em *log* com uma estratégia de RAID distribuído – os segmentos de *log* são subdivididos em seções com dados de correção de erro e escritos nos discos em diferentes nós de rede. É obtido um desempenho de quatro a cinco vezes o do NFS para escrita de arquivos grandes, com menores ganhos para arquivos curtos.

Novas estratégias de projeto • A disponibilidade de redes de alto desempenho (como a ATM e a Gigabit Ethernet) tem estimulado vários esforços para fornecer sistemas de armazenamento persistente que distribuem dados de arquivo de maneira altamente favorável à escalabilidade e à tolerância a falhas entre muitos nós em uma intranet, separando as responsabilidades da leitura e da escrita de dados das de gerenciar os metadados e atender às requisições dos clientes. A seguir, esboçamos dois desses desenvolvimentos.

Essas estratégias escalam melhor do que os servidores mais centralizados que descrevemos nas seções anteriores. Geralmente, elas exigem um alto nível de confiança entre os computadores que cooperam para fornecer o serviço, pois incluem um protocolo de nível mais baixo para a comunicação com os nós que contêm dados (algo muito parecido com uma API de “disco virtual”). Portanto, sua abrangência provavelmente é limitada a uma única rede local.

xFS: um grupo da Universidade da Califórnia, em Berkeley, propôs uma arquitetura de sistema de arquivos de rede sem servidor e desenvolveu um protótipo de implementação, o xFS [Anderson *et al.* 1996]. Sua estratégia foi motivada por três fatores:

1. a oportunidade oferecida pelas redes locais rápidas de permitir que vários servidores de arquivos transfiram grandes volumes de dados de forma concorrente para os clientes;
2. a maior demanda de acesso a dados compartilhados;
3. as limitações fundamentais dos sistemas baseados em servidores de arquivos centralizados.

A respeito de (3), eles se referem ao fato de que a construção de servidores NFS de alto desempenho exige *hardware* relativamente caro, com várias CPUs, controladoras de discos e de rede, e que existem limites para o processo de particionamento do espaço de arquivos – a distribuição de arquivos compartilhados em sistemas de arquivos separados, montados em diferentes servidores. Eles também apontam para o fato de que um servidor central representa um único ponto de falha.

O xFS é sem servidor, no sentido de que ele distribui as responsabilidades de processamento de um servidor de arquivos para um conjunto de computadores disponíveis em uma rede local, com uma granularidade de arquivos individuais. As responsabilidades de armazenamento são distribuídas independentemente do gerenciamento e de outras responsabilidades de serviço: o xFS implementa um sistema de armazenamento RAID em *software*, dividindo dados de arquivos entre discos de vários computadores (a esse respeito, ele é um precursor do Tiger Video File System, descrito no Capítulo 20), junto a uma técnica de estruturação de *log* de uma maneira semelhante ao sistema de arquivos Zebra.

A responsabilidade pelo gerenciamento de cada arquivo pode ser alocada para qualquer um dos computadores que suportam o serviço xFS. Isso é obtido por meio de

uma estrutura de metadados chamada de *mapa de gerência*, que é replicada em todos os clientes e servidores. Os identificadores de arquivo incluem um campo que atua como índice no mapa de gerência, e cada entrada no mapa identifica o computador correntemente responsável por gerenciar o arquivo correspondente. Várias outras estruturas de metadados, semelhantes àsquelas encontradas em outros sistemas de armazenamento estruturado em *log* e RAID, são usadas para o gerenciamento do armazenamento de arquivos estruturado em *log* e no armazenamento em tiras de discos.

Foi construído um protótipo preliminar do xFS e seu desempenho foi avaliado. O protótipo estava incompleto no momento em que a avaliação foi realizada – a implementação da recuperação de falha estava inacabada e o esquema de armazenamento estruturado em *log* não tinha um componente limpador para recuperar o espaço ocupado por *logs* antigos e arquivos compactados.

As avaliações de desempenho realizadas com esse protótipo usaram 32 Sun SPARCstations, com um e com dois processadores (*dual*), conectados em uma rede de alta velocidade. As avaliações compararam o serviço de arquivo do xFS executando em até 32 estações de trabalho com o NFS e o AFS executando, cada um, em uma máquina *dual*. As larguras de banda de leitura e escrita obtidas com o xFS, usando 32 servidores, ultrapassaram as do NFS e do AFS, em um único servidor de processador *dual*, por um fator de aproximadamente dez. A diferença no desempenho foi muito menos pronunciada quando o xFS foi comparado com o NFS e o AFS usando o *benchmark* padrão do AFS. Porém, de modo geral, os resultados indicam que a arquitetura de processamento e armazenamento altamente distribuída do xFS apresentam uma tendência promissora para a obtenção de uma melhor escalabilidade em sistemas de arquivos distribuídos.

Frangipani: o Frangipani é um sistema de arquivos distribuído com alta escalabilidade, desenvolvido e implantado no Digital Systems Research Center (agora, Compaq Systems Research Center) [Thekkath *et al.* 1997]. Seus objetivos são muito parecidos com os do xFS e, assim como o xFS, ele os aborda com um projeto que separa as responsabilidades de armazenamento persistente das outras ações de um serviço de arquivos. No entanto, o serviço do Frangipani é estruturado como duas camadas totalmente independentes. A camada inferior é fornecida pelo sistema de disco virtual distribuído Petal [Lee e Thekkath 1996].

O Petal fornece uma abstração de disco virtual distribuído sobre os discos localizados em vários servidores, em uma rede local. A abstração de disco virtual tolera a maioria das falhas de *hardware* e de *software* com a ajuda de réplicas dos dados armazenados e harmoniza, por meio do reposicionamento dos dados, a carga de forma autônoma nos servidores. Os discos virtuais do Petal são acessados por meio de um *driver* de disco UNIX, usando operações de entrada e saída de blocos padrão, de modo que eles podem ser usados para suportar a maioria dos sistemas de arquivos. O Petal acrescenta entre 10 e 100% na latência de acessos a disco, mas a estratégia de uso da cache resulta em desempenhos de saída de leitura e escrita pelo menos tão bons quanto os das unidades de disco locais.

Os módulos servidores do Frangipani são executados dentro do núcleo do sistema operacional. Assim como no xFS, a responsabilidade por gerenciar arquivos e as tarefas associadas (incluindo serviço de travamento de arquivos para os clientes) é atribuída dinamicamente aos nós, e todas as máquinas enxergam um espaço de nomes de arquivo unificado, com acesso coerente (com semântica aproximada à da cópia única) para os arquivos compartilhados que podem ser atualizados. Os dados são armazenados em um formato estruturado em *log* e em tiras, no disco virtual Petal. O uso do Petal desobriga o Frangipani da necessidade de gerenciar espaço físico em disco, resultando em uma implementação de sistema de arquivos distribuído muito mais simples. O Frangipani pode simular as interfaces de vários serviços de arquivos existentes, incluindo o NFS e o DCE/DFS. O desempenho do Frangipani é pelo menos tão bom quanto o da implementação da Digital do sistema de arquivos UNIX.

12.6 Resumo

Os principais problemas de projeto para sistemas de arquivos distribuídos são:

- o uso eficiente de cache no cliente para obter um desempenho igual ou melhor do que o dos sistemas de arquivos locais;
- a manutenção da consistência entre várias cópias de arquivos de cliente colocadas em cache, quando essas cópias são atualizadas;
- a recuperação após uma falha do cliente ou do servidor;
- alto desempenho de saída para leitura e escrita de arquivos de todos os tamanhos;
- capacidade de mudança de escala.

Os sistemas de arquivos distribuídos são bastante empregados em sistemas computacionais institucionais, e seu desempenho tem sido o assunto de muita otimização. O NFS possui um protocolo sem estado (*stateless*), mas tem mantido, com a ajuda de alguns aprimoramentos relativamente pequenos no protocolo, sua posição inicial como tecnologia de sistema de arquivos distribuído dominante contando com implementações otimizadas e suporte de *hardware* de alto desempenho.

O AFS demonstrou a exequibilidade de uma arquitetura relativamente simples, usando o estado do servidor para reduzir o custo da manutenção de caches de cliente coerentes. O AFS supera o NFS em muitas situações. Avanços recentes têm empregado a disposição de dados em tiras (*striping*) entre vários discos e escrita estruturada em *log* para melhorar ainda mais o desempenho e a capacidade de mudança de escala.

Os sistemas de arquivos distribuídos mais modernos têm alta escalabilidade, fornecem bom desempenho entre redes locais e remotas, mantêm a semântica de atualização de arquivos com cópia única e toleram falhas e se recuperam delas. Os requisitos futuros incluem o suporte para usuários móveis com operação desconectada, e reintegração automática e garantia de qualidade do serviço para satisfazer a necessidade de armazenamento persistente e a distribuição de fluxos de dados multimídia e outros fluxos dependentes do tempo. As soluções para esses requisitos serão discutidas nos Capítulos 18 e 20.

Exercícios

- 12.1 Por que não existe nenhuma operação *open* ou *close* em nossa interface para o serviço de arquivos simples ou para o serviço de diretório? Quais são as diferenças entre nossa operação de serviço de diretório *Lookup* e a operação *open* do UNIX? páginas 532–534
- 12.2 Descreva, em linhas gerais, os métodos pelos quais um módulo cliente poderia simular a interface de sistema de arquivos UNIX usando nosso modelo de serviço de arquivos. páginas 532–534
- 12.3 Escreva uma função *PathLookup(Pathname, Dir) → UFID* que implemente a operação *Lookup* para nomes de caminho do tipo UNIX, com base em nosso modelo de serviço de diretório. páginas 532–534
- 12.4 Por que os UFIDs devem ser exclusivos entre todos os sistemas de arquivos possíveis? Como a exclusividade dos UFIDs é garantida? página 535
- 12.5 Até que ponto o Sun NFS se desvia da semântica de atualização de arquivo com cópia única? Construa um cenário no qual dois processos usuário compartilhando um arquivo funcionariam corretamente em um único computador UNIX, mas observariam inconsistências ao serem executados em computadores diferentes. página 542
- 12.6 O Sun NFS pretende suportar sistemas distribuídos heterogêneos por meio de um serviço de arquivo independente do sistema operacional. Quais são as principais decisões que o de-

- envolvedor de um servidor NFS para um sistema operacional que não seja o UNIX teria de tomar? Quais restrições um sistema de armazenamento deve obedecer para ser conveniente para a implementação de servidores NFS? *página 536*
- 12.7 Quais dados o módulo cliente NFS deve conter sobre cada processo usuário? *páginas 536–537*
- 12.8 Descreva, em linhas gerais, as implementações de módulo cliente para as chamadas de sistema UNIX *open()* e *read()*, usando as chamadas de RPC do NFS da Figura 12.9, (i) sem e (ii) com uma cache no cliente. *páginas 538, 542*
- 12.9 Explique por que a interface RPC das primeiras implementações do NFS é potencialmente insegura. A brecha de segurança foi fechada no NFS 3 pelo uso de criptografia. Como a chave de criptografia é mantida em segredo? A segurança da chave é adequada? *páginas 539, 544*
- 12.10 Após o *timeout* de uma chamada de RPC para acessar um arquivo em um sistema de arquivos montado incondicionalmente, o módulo cliente NFS não retorna o controle para o processo usuário que originou a chamada. Por quê? *página 539*
- 12.11 Como o *automounter* do NFS ajuda a melhorar o desempenho e a capacidade de mudança de escala do NFS? *página 541*
- 12.12 Quantas chamadas de *Lookup* são necessárias para solucionar um nome de caminho de cinco partes (por exemplo, */usr/users/jim/code/xyz.c*) para um arquivo armazenado em um servidor NFS? Qual é o motivo para realizar a transformação passo a passo? *página 540*
- 12.13 Qual condição deve ser atendida pela configuração das tabelas de montagem nos computadores clientes para que a transparência de acesso seja obtida em um sistema de arquivos baseado no NFS? *página 540*
- 12.14 Como o AFS obtém o controle quando é executada, por um cliente, uma chamada de sistema de abertura (ou fechamento) referindo-se a um arquivo no espaço de arquivo compartilhado? *página 549*
- 12.15 Compare a semântica de atualização do UNIX ao acessar arquivos locais com as do NFS e do AFS. Sob quais circunstâncias os clientes poderiam conhecer as diferenças? *páginas 542, 554*
- 12.16 Como o AFS lida com o risco de perda de mensagens de *callback*? *página 552*
- 12.17 Quais características do projeto do AFS o tornam mais escalável que o NFS? Quais os limites para sua escalabilidade, supondo que servidores podem ser adicionados conforme for exigido? Quais desenvolvimentos recentes oferecem maior escalabilidade? *páginas 545, 556, 561*